TECHNISCHE HOCHSCHULE MITTELHESSEN

CAMPUS GIESSEN

MNI
Mathematik, Naturwissenschaften und Informatik

Master Thesis

Reengineering and analysis of a model interpreter implementation in the domain of mobile applications

zur Erlangung des akademischen Grades

Master of Science

eingereicht im Fachbereich Mathematik, Naturwissenschaften und Informatik an der Technische Hochschule Mittelhessen

von

Samuel Schepp

May 29, 2019

Referent: Prof. Dr. Peter Kneisel

Korreferent: Dennis Priefer

Erklärung der Selbstständigkeit

Hiermit versichere ich, die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt sowie die Zitate deutlich kenntlich gemacht zu haben.

Gießen, May 29, 2019 Samuel Schepp

This thesis describes configuration-based mobile applications and their infrastructure. In addition, a possible way of implementing a configuration system is discussed. Finally, the implementation is compared to the approach of established literature by classifying the model and interpreter.

Furthermore, there is an evaluation of the use case of interpreting and generated apps.

Contents

1 Introduction 1
- 1.1 Problem definition . . . 1
- 1.2 Motivation . . . 2
- 1.3 Limitation . . . 3
- 1.4 Structure . . . 3
  - 1.4.1 Logical Approach . . . 4
  - 1.4.2 Goals . . . 4

2 Background 5
- 2.1 Company . . . 5
  - 2.1.1 Markets . . . 5
  - 2.1.2 City Marketing . . . 6
  - 2.1.3 Problem . . . 6
- 2.2 Model Interpreter System . . . 8
  - 2.2.1 Use Cases . . . 9
  - 2.2.2 Interoperability . . . 9
- 2.3 Models . . . 10
  - 2.3.1 General Model Theory . . . 10
  - 2.3.2 Specialized Model Theory . . . 11
  - 2.3.3 Model Representation . . . 11
- 2.4 Model-Driven Architecture . . . 12
- 2.5 Model-Driven Development . . . 14
  - 2.5.1 Modeling a Software System . . . 15
  - 2.5.2 Meta Programming . . . 16
  - 2.5.3 Code Generation . . . 17
- 2.6 Model Interpreters . . . 22

3 Concept 25
- 3.1 Implementation Goals . . . 25
  - 3.1.1 Functional Model Requirements . . . 25
  - 3.1.2 Non-Functional Model Requirements . . . 27
- 3.2 Model Syntax . . . 28
  - 3.2.1 Drawbacks . . . 29

3.2.2 Iterative Development Process . . . 30
3.3 Client Application . . . 31
3.3.1 App Configuration . . . 31
3.3.2 App Template . . . 32
3.3.3 App Bundle . . . 32
3.3.4 Functional Interpreter Requirements . . . 33
3.3.5 Non-Functional Interpreter Requirements . . . 34
3.3.6 Implementation Weak Points . . . 35
3.4 Native Approach . . . 35
3.5 Virtual Machine Approach . . . 36
3.5.1 Native Components . . . 37
3.5.2 Virtual View Hierarchy . . . 38
3.5.3 Shared Code . . . 39
3.5.4 Testability . . . 39

4 Implementation 41
4.1 Initial State . . . 41
4.2 Native Approach Implementation . . . 42
4.2.1 Native User Interface . . . 42
4.2.2 Concurrency . . . 43
4.2.3 Testability . . . 43
4.3 Virtual Machine Implementation . . . 44
4.3.1 Base Elements . . . 44
4.3.2 Change Management . . . 46
4.3.3 Concurrency . . . 49
4.3.4 Proof of Concept . . . 50
4.3.5 Testability . . . 54
4.3.6 Core Rules . . . 56
4.3.7 Maintainability . . . 58

5 Evaluation 61
5.1 Meta Model . . . 61
5.1.1 Development Approach . . . 62
5.1.2 Structure . . . 62
5.2 Architecture Analysis . . . 63
5.2.1 Native Approach . . . 63
5.2.2 Virtual Machine Approach . . . 63
5.3 Requirement Analysis . . . 63
5.3.1 Native Approach . . . 63
5.3.2 Virtual Machine Approach . . . 63

6 Conclusion 65
6.1 Summary 65
6.2 Discussion 65
6.3 Next Steps 65
6.3.1 Hybrid Approaches 65
6.3.2 Runtime Models 65
6.3.3 Abstraction 65

Bibliography 67

Index of abbreviations 69

List of Figures 71

List of Tables 73

Listings 75

1 Introduction

Mobile application and digital media usage is still on the rise[Per]. ”Apps” are one of many possible ways to interact with customers and consumers. A web appearance on social media or a dedicated web page can also fulfill these needs.

A mobile application can enable user specific features without logging in, because the user can be identified by the installed instance, that is unique for every user. It is possible to activate special features like geo-fencing, push notifications or local data storage, that is synchronized with the platform specific cloud services (e.g. Apple iCloud or Google Drive), without any custom user authentication.

A mobile app is integrated more deeply into the device, than a web app. However, web browsers already support a lot of the features by now, that originally were only available to native apps[1]. The current development indicates, that the differences will be further eliminated by the browser vendors.

1.1 Problem definition

Mobile applications have significant disadvantages, that push the development of web apps forward. Changes have to be approved by the phone vendors, in order to be able to release updates to the app. This causes development delays and reduces the ability to react to events or customer requirement changes.

All apps have to be kept in good condition, which creates more work, the more apps there need to be maintained. New operating system features have to be implemented in all of the apps, which can cause inconsistencies. All apps have to be tested in detail, to make sure that all apps implement the required change correctly. Platform changes can only be dealt with one by one. Developers need deep system knowledge to develop the apps. Every change has to go through one of each platform experts, so development is expensive.

1 E. g. Local storage, push notifications. These features are now part of many browser’s APIs.

An example for a change, that all apps should implement is the display notch. It allows the screen to extend to the edge of the display. However some of the screen is cut out to make room for sensors and the camera.

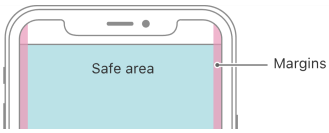

Figure 1.1: Display notch with safe area.[1]

The notch is integrated to the default interface framework, so that developers can react accordingly and use the provided margins. This change requires all apps to implement a correct way to handle that. Developers have to go though every screen and correct the view.

1.2 Motivation

To be able to deliver the full mobile application experience, the app has to be native. This will change, when all of the phone's hardware interface is exposed to the web browser and user interface performance is better. But for now, the only way to use these features and be good on battery life and data usage, is by providing a native app, that is written in the native language and uses features, directly provided by the phone vendors.

Web app wrapper like "ionic" are able to use these features and provide their own interface to the web app. In this way, a web app can use restricted components like push notifications and beacons.

With this approach, every new app still has to be developed and old ones have to be maintained individually. In addition, it involves the dependence on third party framework vendors. Building apps with ionic creates a massive list of dependencies: ionic, angular, npm and nodejs. New operating features have to be implemented by the framework maintainers, which can cause massive delays and could completely prevent the implementation of new features. Native apps have the ability to reduce dependence, by only using native libraries, which are very comprehensive by default. New features are available immediately, since the native frameworks introduce them.

To be able to create many apps with little effort, a model-driven approach can be considered. A well abstracted model can have the ability to be easy to understand by

1 Source: Apple Human Interface Design Guidelines 2018 https://developer.apple.com/design/human-interface-guidelines/ios/visual-design/adaptivity-and-layout/

developers and the platforms can bring their respected implementation in, so any app looks and feels native. A service can deliver pre-rendered sub-models, so that complex client workload can be moved to the server.

Existing apps are easy to maintain, because there is only one code base per platform. One bug fix applies to all apps. Also, it is easy for anyone to change small things like a label, a background color or the sorting of a list. The goal is, to be able to configure new apps in one day, together with the customer.

1.3 Limitation

This thesis will not describe the development experience nor the initial expense of the project. Neither will this thesis evaluate financial aspects and at which point, the project is worth the development costs. The domain examination will be narrowed down to the two main ones, mobile phones and digital signage. Relevant aspects of model-driven software development and interpreters are examined and applied to the discussed system.

1.4 Structure

The thesis will give some information on the background of the interpreter. The company Fabrik19 GmbH, which is actively developing and maintaining an implementation, will be described and characterized. After that, the general concept of model-driven development, code generators and interpreters are examined and presented.

The main content will then be about the concrete implementation, by describing the most significant design choices, overall infrastructure and usage examples. The focus will be on the two main platforms 'iOS' and 'Android'. The web and *.NET* clients are not considered, because they are not targeted for mobile devices and only support a subset of the model.

In the final chapter, the concept is applied to the implementation, to see how it fits the theoretical model of interpreters. Differences are worked out and discussed. Also, an outlook will be given, where additional topics, like hybrid approaches, runtime models and abstraction are covered.

1.4.1 Logical Approach

To be able to evaluate the interpreter, requirements are listed and described. These will be used to validate the implementation.

Since the implementation for the iOS platform has been re-engineered to meet current demands, the issues with the old version will be outlined and compared to the new one.

1.4.2 Goals

This thesis will show how a model interpreter can be evaluated against requirements and informal criteria. In addition, the re-engineering and model development processes will be compared to common approaches in modern literature.

The following questions will be answered by applying theoretical approach to the given example.

> *Which requirements are placed on the model interpreter implementation?*
>
> *How can the specific model interpreter be implemented?*
>
> *How can the implementation be evaluated on the basis of the requirements?*

2 Background

The system, that is discussed here, is a model interpreter that is designed to run mobile applications on iOS and Android. The system, which includes the interpreters and the backends are called mos[1]. It is in active development and powers over 50 apps that all are available in the Apple AppStore and Google PlayStore. The app categories range from retail to event to city marketing apps[2].

2.1 Company

The creator of the system is Fabrik 19 GmbH. The company is based in Gießen, Hesse, Germany and was founded 2013 by Mark Pralle and Sven Herchenhein. At the moment, the company has bout 25 employees. Besides of apps, there are also other projects and solutions, like consulting and web hosting.

2.1.1 Markets

Fabrik19 does not deliver apps directly to customers, but is partner of retail corporations and city marketing agencies. The current flagship is the Gießen App (figure 2.1), which was developed in cooperation with DISTAMA GmbH. The app was downloaded 34,000 times and has 20 distinct and active users per day.

Besides mobile apps, digital signage is another important market. The companion to the Gießen App is a stele that contains a large touchscreen and runs an app that is optimized for big touchscreens. It is located on the main pedestrian shopping street in Gießen and provides respective information on the stores and points of interests[3]. Other stele systems are also active in Globus hardware stores and Globus grocery stores.

1 Mobility Suite
2 https://fabrik19.de/technologie/
3 https://distama.de/bid-seltersweg/

Figure 2.1: Gießen App, Marketing showcase image. Source: Giessen Entdecken http://giessen-entdecken.de/die-app/

2.1.2 City Marketing

Since the apps use live data and have the ability to change it's content structure, an active support and maintenance is needed to keep the app alive.

To be able to advertise and publish city marketing apps as a service, Distama GmbH was founded which is a subsidiary firm of Fabrik19. The goal was to combine technology knowledge with marketing, to be able to provide the app and the attached customer support. City marketing agencies are not always able to support an app, because they lack experience and domain knowledge. Distama is the mediator between the developer, the customer and the user.

2.1.3 Problem

To be able to deploy dynamic apps and change it's contents live, a model-driven interpreter architecture is being used. The contents and behavior are described by modeling the app as an abstract configuration. In addition, the client application is developed to reload it's contents and configurations if changed. This way, an application that has been deployed one time will always be up to date. This is needed for mobile applications but more importantly for digital signage installations. That is way, this organization chose a model-driven development approach for it's mobile applications.

Figure 2.2: City marketing stele for shapping streets. Source: Fabrik19 Portfolio https://fabrik19.de/project/interaktive-stele-giessener-seltersweg/?lang=en

This can be applied to similar situations, where the requirement is to produce many application with similar structure, or where behavior and implementation has to be separated, because of complexity. One example is the generation of C-code for embedded systems using Simulink[1]. It allows designing and modeling algorithms and structures by setting up a graphical block diagram. It is possible to structure algorithms in systems and subsystems, which behave as a block themselves. It is then possible to run simulations on those models, which is not as simple on the native code. Platform specific C-code can then be generated.

Another example are video games or augmented reality applications, where there is a strict separation of content and rendering. Content developer are able to artistically create the worlds, while engine experts are available to maintain the rendering and hardware specific optimizations.

Those two examples show, that model-driven development is used in several domains and it is a possible solution to manage complexity. Additional benefits like the ability to switch to another model without changing the app are the core ideas why the Fabrik19 organization chose this approach.

1 https://www.mathworks.com/products/simulink.html

2.2 Model Interpreter System

The main development focus lays on the app interpreter platform ”mos”. It supports the two major platforms iOS and Android, but there is also a lightweight version available in .Net, which is running on the steles as a Windows app. A web client is also actively in development, which aims to replace the .Net version.

A configuration are multiple JSON files, that represent the app meta information, its layout and its behavior. The files are called node. There is one root node, whose URL[1] is baked into the app while compiling. This root node defines the first screen of the app. All the other screens can be defined in other nodes. When displaying another node, the new node is loaded from cache or network and its new view is pushed on the view stack, which is done by the interpreter. On the particular platforms, this action will look different and will use the platforms native animations. Web apps like Ionic have their own push animations which makes them feel foreign. The previous node can pass data, like the ID of the item that should be displayed.

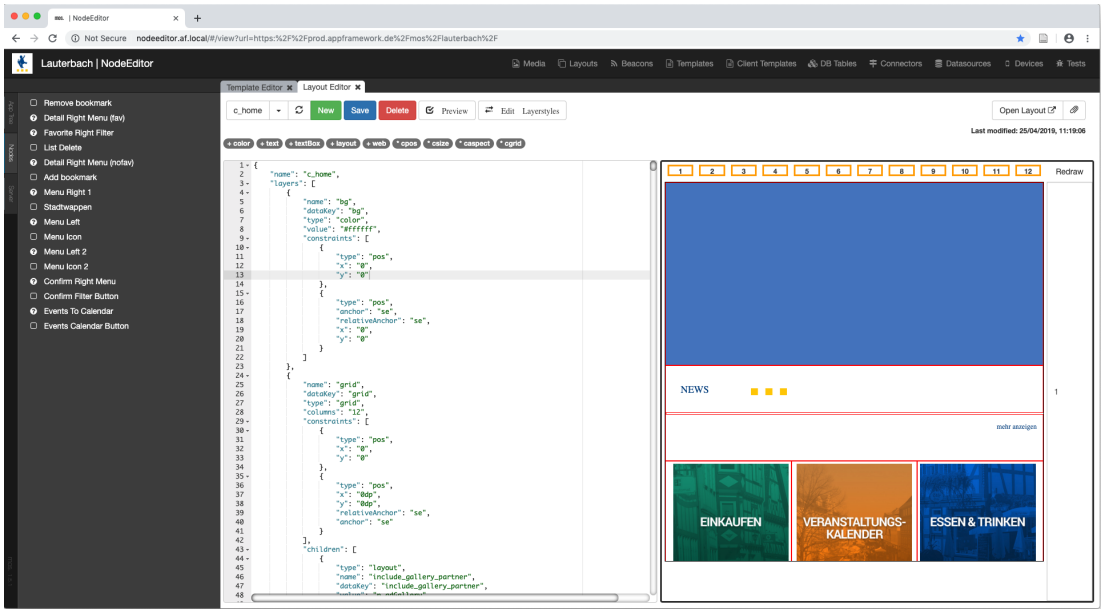

Figure 2.3: The web based node editor is used to develop and edit the models. The Lauterbach App landing page is displayed.

The goal is, to be able to create a new app in one day, by meeting with the customer and configuring the app together. The syntax of the configuration is designed to abstract all of an apps standard behavior. Because of a custom layout system, that can be described with constraints and will layout automatically, it is not possible to recognize that it is only a configured app, instead of a custom native one.

1 Unified Resource Locator

The apps have sophisticated caching and optimization mechanisms, which makes it possible to use them completely offline, without any additional configuration effort. The configuration, images and data are cached heavily, which makes the apps very fast. But because of the dynamic design of the configuration syntax, it is possible to clear the caches and force new versions. The cache first policy of the interpreter makes sure, that contents are always available and loads changes in the background.

Besides of displaying data, the client is highly intractable and can perform various custom tasks on the device, like uploading local data, saving data to the devices user settings and providing push, beacon and location infrastructure.

This data can be used in describing the layout by defining layer conditions, which makes the app behave accordingly.

2.2.1 Use Cases

The main use cases are marketing and customer interaction. The concept is data source driven and enables configuration of any data source. Because of a template engine, data can be arranged for display in an intuitive way.

Since the configuration can be changed on the fly and without the need of providing an update via app store, the contents can be adjusted to fit current events like holidays or promotions. Especially for digital signage it is important to be able to change the contents without any application updates.

Any change to the configuration is reflected as soon as the app checks for configuration changes. The update mechanism are also highly customizable, so that the optimal update intervals and strategies can be used for different apps.

A city marketing app might need a medium update interval, so that promotions and special content are available when needed. A event app should have a very short update interval, so that the events program changes or announcements are delivered as fast as possible.

2.2.2 Interoperability

The app can be filled with data via common REST services, that deliver JSON data. The app uses HTTP mechanisms and a custom delta system to only load new data. The mos backend system also has a data proxy, which can load any data and provide it in a simpler, preprocessed format, so it is simple for the apps to download. This way, it

is possible to use any data source and still be able to change the templates and layouts without updates.

The apps have the capability to be integrated in a Wordpress instance, a chat service like Mattermost or an secured API[1] that uses authentication methods like client certificates, authorization headers or JSON web tokens.

By default any network communication is secured with an app specific client certificate. Other authentication methods have to be configured.

2.3 Models

Since the mos-Framework tries to abstract mobile application platforms into a common language, the configuration could be considered to be an abstract model of a mobile application. To classify and evaluate the model approach for this domain, the following section will discuss models and their properties.

2.3.1 General Model Theory

A general model is described by Herbert Stachowiak[2]. He defines three properties, that apply to all models.

Abbildungsmerkmal (Representation): A model is a representation of an original, that is either a natural or artificial thing, that can have properties and behavior. The original can be a model it's self.

Verkürzungsmerkmal (Reduction): The second property is the ability to reduce properties and behavior of the original to a set, that is relevant for the model handlers. The original might be a complex system and only a subsystem or specific properties and their interaction are relevant. For this, the model can reduce the complexity, to be more simple to understand and to minimize the error rate.

Pragmatisches Merkmal (Pragmatism): Models can not be clearly assigned to their originals. That means, that original and model are not equal, but can be interchanged. There are three cases defined by Stachowiak, in which the model can be used as an replacement. Models are not only models of something, but they are also models for someone, a person or an artificial model user. They perform their functions within a

1 Application Programming Interface
2 [Staa] 131

time interval. And they are models for a specific purpose. Often times, the purpose is an action that is not possible on the original because it may be to complex, or an physical original could be destroyed.

If theses properties apply an object, i can be called model according to Stachowiak. It can be applied by numerous of common usages of the word. Mathematical models, philosophical models, simulations and concrete object models are some examples, that fit the requirements.

2.3.2 Specialized Model Theory

To classify models in a more specific way, they can be categorized as descriptive and prescriptive models. A descriptive model represents an existing system, as it describes it's current behavior. The general model specifications can still be applied, since all three properties are true for such a model. It cannot be modified unless the original changes, since it is representing it directly. A change to one of them without updating the other would result in the model not representing the original, which makes it a prescriptive model. A change on the original which is not relevant to the model will keep its validity.

Models, that represent a fictive object or system are called prescriptive models. The prefix "pre-" indicates, that the model exists before the original. The model is then used to prototype a system, and test their behavior and properties, without the need to create the full system.

According to Seidewitz 2003 [Sei] both descriptive and prescriptive models define a set of originals. That means, that a original can be derived from a model by adding additional information that does not contradict each other. Often times, the information that must be added is trivial or can be fetched from somewhere else. That is possible, because the model has by definition all of the necessary information.

A derived original fulfills all of the models properties, otherwise the original is not a correct derivation.

2.3.3 Model Representation

It is common to represent models either by text or using graphical editors. Both are suitable for their respective use cases. If it is advantageous, that objects and their relationships can be arranged on a two dimensional surface, a graphical can be used. If the model syntax is similar to a programming language, graphic editors might obstruct the workflow. A text-based representation is commonly used in software development,

so it is easy to integrate models into an existing infrastructure, because for example, it is easy to manage versioning using a VCS[1]. Also, tools for developing DSLs[2] or generating code mostly operate on a text based model representation.

2.4 Model-Driven Architecture

To handle evolving infrastructure, changing requirements and technology development, a model-driven architecture (MDA[3]) standard has been developed by the *Object Management Group*[4] [Mil]. The goals are to lower costs and reduce complexity of a software's maintenance. The OMG[5] noticed, that the requirements of a software system is much more than only the product itself. In addition, interoperability, changes to the environment and data migration require constant maintenance and continuous progression. Since classic applications are written in a platform dependent programming language, those task tend to be very costly. To be able to produce software that is build to last, easy to integrate, easy to change and able to keep up with new technology, they defined a standard on how to model a system. Models separate the implementation details that are required to run in a certain environment from the actual way of using the platforms capabilities ([Mil] 2.1.2). This model is in a certain level of abstraction and can be transformed to platform specific models and therefor executable code. Now, with the model as input, parts of the application can be generated. If requirements change, the model which is human readable can be changed and new code be generated instantly. If the platform changes or new technology like a different data storage or programming language occurs, the tool chain can be updated and all of the generated code can be ge-generated to reflect the current technology state. This leads to long term flexibility of implementation details, integration capabilities, maintenance complexity and simulation ([Mil] 1.2).

There are three levels of abstraction defined by MDA ([Mil] 2.2.11).

- **Platform Specific Model**

  A model defines, how platform specific functionality is used. An example is a *Jenkinsfile*, that models a pipeline of processes that can be run on a Jenkins build service. It is a model, because there is no concrete code implemented, but instead jobs are modeled by using predefined task definitions and a stage structure. It is

1 Version Control System
2 Domain Specific Languages
3 Model Driven Architecture
4 https://www.omg.org/
5 Object Management Group

platform specific, because Jenkins-specific functionality can be used like defining build status or using plugin interfaces.

- **Platform Independent Model**

  The model is suitable for more than one specific platform. This is achieved by defining a set of functionality that is implemented by multiple platforms. A typical example are UML[1] class or activity diagrams, because they define a structure, without relying on a platform. Many platforms support module and class access modifier, so that public, protected and private can easily be mapped to many other platforms. As a consequence PIMs[2] can be mapped to PSMs[3].

- **The Computation Independent Model**

  Not only is this kind of model independent to a platform, but also to the way the result is achieved. This is typical for declarative models and allows the most freedom for the generator or interpreter. An SQL[4] statement can be considered computation and platform independent, since it can be used on multiple SQL based platforms and it does not model the way the result is evaluated. Some may argue, that SQL is to low level in some contexts and in fact does describe the way of operating on data. This point of view depends on the models context. If the SQL statements are directly used by handwritten code, it can be considered computation independent. But if it is part of generated software system, it is considered to be a portion of the implementation details.

Those three level can wrapped by two additional levels. [Jea] 3 defines the level of *Requirements Engineering*, which is the most abstract level and code level which is the lowest level of abstraction. By *lowest level*, the author references the lowest human intractable level of code, which can be an object oriented language like Java for example. This results in the following abstraction hierarchy:

1. Code
2. Platform Specific Model
3. Platform Independent Model
4. The Computation Independent Model
5. Requirements Engineering

1 Unified Modeling Language
2 Platform Independent Models
3 Platform Specific Models
4 Structured Query Language

2.5 Model-Driven Development

Models can be used in a software development process (MDSD[1]) or as part of a software architecture (MDA). Software is also a model, which represents real life objects and procedures. Since it is well structured by default, it is fairly easy to create model from a existing code basis. UML is a widely used modeling languages, which can represent static, dynamic and interactive aspects of software systems.

The problem is, that the domain specific structure of the code can be hard to understand, because the abstraction level of a modern object oriented programming language is much lower, than the domain requires. VÖLTER and STAHL describe the result of scattering relevant information in the code as: 'The much-treasured inner structure is present in a cloudy, distributed, and of course also a strongly individualized form.'[2] That means, that the domain's structure, components and behavior definition is represented in the software code, but in a way that is hard to understand. That is, because the relevant parts and the domain critical definitions are spread in the software's code. It is also harder to unterstand a domains properties, by only reading the programming language.

A software model, like an UML class diagram are an old tool to document and model software systems. The problem here is, that the abstraction level of an UML class diagram is often times the same as the programming language itself. A developer could find himself transforming a class into a diagram or generating a diagram, which does not contribute to the understanding of the domain structure. This happens, because the programming language itself is abstracted, but not the domain itself.

To address the problem of widely spread domain implementations, the programs important and relevant structure is compressed in a model, which defines and represents the program on a abstract and formal level. The model can be a textual DSL, a graphical diagram or even an informal text, that is then converted into a concrete syntax. It contains all the domain relevant information, which is required to derive a full implementation from it.

This procedure results in a model becoming part of the source of a program, and not only the documentation. A good model syntax can lead to the model being the documentation itself.

1 Model Driven Software Development
2 [Stab] 14

2.5.1 Modeling a Software System

An existing implementation can be analyzed, to create a model from it. If no model infrastructure exists, this is the way to approach the transformation.

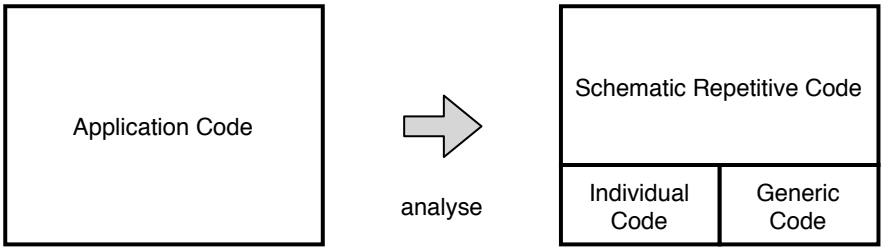

Figure 2.4: Analyzing an existing implementation (according to [Stab] 15).

Figure 2.4 shows, how the code basis of an existing implementation of a software system can be split into three categories[1].

The **schematic repetitive code** is the part, that needs to be abstracted into a model. It contains the application specific logic, that is later modeled by a user. It does not contain code, that is equal for all applications in the same domain.

Generic code is derived from the domain and target platform and not from the concrete application. Therefore, it is not modeled, but generated by the domain's MDD[2] infrastructure. It is mostly boilerplate code, that should not be part of the model. It is the same for every application and is later part of the generator templates.

Individual code is unable to be represented in the model, because it is to specific and does not fit into the domain. The tools need to make sure that this code can be integrated into the generated code later. To simplify the MDSD process, individual code should be avoided as far as possible, because in the worst case the individual code makes up all of the application code, which eliminates the possibility to use a model to represent the application. In the best case, the individual code portion is empty. This allows for a more simpler tools, because the application can be generated completely out of the model and there is no need for individual code interfaces.

Figure 2.5 shows, how the analyzed parts can now be separated into source code and steady code. Source code describes the part of the code that is application specific and is developed by the MDD user. The application model and individual code should be checked into source control, since that is the actual source of the application, given a MDD infrastructure exists.

1 [Stab] 15
2 Model Driven Development

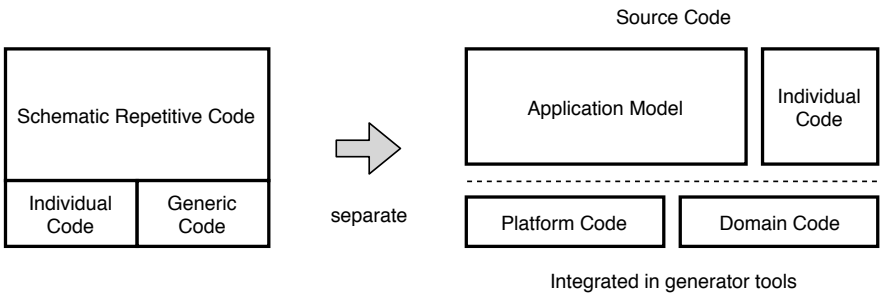

Figure 2.5: Separation of code into source code and steady code (according to [Stab] 15).

Platform and domain code is part of the MDSD tools. It is baked in, so that it is part of the output while generating. In case of code generators, this code would be part of the templates. In theory, it is now possible to develop any app, that fits into the domain, by only defining the model and the individual code. Extensions of the model's capabilities might need to be reflected in the platform or domain code.

2.5.2 Meta Programming

Now that there is a model of an application, a concrete implementation has to be created from the model. Code generators can take the model and generate a software system, that is an instance if the model. Another approach are model interpreters, which execute the model directly, similar to a virtual machine.

Code generation enables advantages, that are available to the generated code. By using meta programming more concepts can be expressed than the target language allows.

Code efficiency

A code generator has the privilege to build highly efficient code, by using the fastest statements, which have the fewest overhead. Modern programming language features like flexible syntax, polymorphism or reflection cause the application to run slower then needed[1]. A code generator can perform polymorphism and reflection on generation time and generate direct code, that is fast. The flexibility now lies in the model and not in the generated code, which tends to be impossible to maintain, because of the optimization potential. Code size can also be reduced, by only creating the needed parts.

1 according to [Stab] 181

Aspects of the applications functionality that normally would be scattered around in the code can be modeled centralized and then generated into the depended locations. An example would be a dependency framework, that is used by many classes. Instead of copy the import code to every class, it can be modeled as an decorator, that will provide the needed boilerplate code to all classes.

Platform language restrictions

Is is possible to break out of the target language's restrictions. The level of abstraction of the meta model and its defined language does not need to match that of the target language. A code generator can generate code that reflects higher level of features. It is possible to generate code that integrates the concept of OOP[1] into a non OOP-language. For example: A generator can generate structs and function pointers in a C/C++ application to imitate objects. The generated code on it's own is not considered safe or structured, but the static context analysis what is done while creating the model performs the sames checks as an OOP-compiler. Another example are generics, that can by used generating untyped objects[2] and perform a static context analysis while generating the code. Any feature that is known from modern languages can be used by imitating the compiler and its static context analysis. In a more extreme case, a virtual machine can be integrated into the application which can run any code.

Generated code with flat hierarchies is easy to analyze statically, because much of the logic is not abstracted into frameworks.

Compatibility

The higher the abstraction level of the meta model, the more platforms can be supported. The static context analysis is independent to the platform and should not integrate platform specific restrictions. A transition to another platform or supporting multiple platforms at once is possible and is one of the core strengths of MDD.

2.5.3 Code Generation

Now that a model exists, code can be generated out of it. Before code can be generated, the model has to be transformed into a generation oriented model. It's purpose is the simplification of the generation process by providing the target structure, which then

1 Object Oriented Programming
2 *Object* (Java), *void∗* (C/C++), *any* (JavaScript)

can be converted to text directly. The transformed model is an model instance, that is targeting a specific platform. In cases of which the model's abstraction level is low and close to the target platforms, the transformation is not needed, since it brings a new level of complexity and abstraction to the process.

A typical example is a high level model, which could be implemented in any imperative language would be transformed into a UML class model. Object oriented programming code can then be generated out of the class model trivially. There are techniques to create program code from a transformed model. The two most popular ones[1] are templates- and API-based code generators.

Template Based Generators

A template is a definition, that is iterating over the model and prints the values on the corresponding locations in a text file. A template mixes the concrete target code presets with a markup notation, that is getting and writing values. The handlebar notation is one of the simplest form of template based code generation. A well formed model, which is considered the input is applied to a template, which results into a resulting output.

The following example generates HTML[2] code by using a XSLT[3]-based templates and XML[4] models. As described in listing 2.1, the model represents a list of text lines.

Listing 2.1: data.xml - The model that is data.

```
<list>
  <entry>A</entry>
  <entry>B</entry>
</list>
```

The template is displayed in listing 2.2. It defines, how the data is transformed into HTML. The template is correct XHTML[5] by itself. This way, a structured and well-defined transformation can be modeled.

Listing 2.2: template.xslt - The templates that transforms data into HTML.

```
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/
    Transform">
  <xsl:template match="/list">
```

1 according to [Stab] 186
2 Hyper Text Markup Language
3 Extensible Stylesheet Language Transformations
4 Extensible Markup Language
5 Extensible Hypertext Markup Language

```
    <ul>
      <xsl:apply-templates select="entry"/>
    </ul>
  </xsl:template>
  <xsl:template match="entry">
    <li>
      <xsl:value-of select="." />
    </li>
  </xsl:template>
</xsl:stylesheet>
```

A PHP[1] script can now trigger the XSLT processor and generate an output file. This output file is shown in listing 2.3.

Listing 2.3: XSLT ouput.

```
<ul>
  <li>A</li>
  <li>B</li>
</ul>
```

The mos interpreter uses XSLT to generate the mandatory license HTML page, which is based on a configuration file. Instead of the full license text, every third party license reference only contains the license type identifier (e.g. MIT). This way, the source file is easy to maintain and extend.

This concrete usage example of XSLT yields several benefits:

1. Changing the style and layout of the HTML file requires only a few changes. Adding or changing the CSS[2] classes of elements only requires to change the one occurrence in the template.

2. The URL of an entry was displayed as the text content of an a element, but also occurs in the $href$ attribute. Changing the URL later, can cause inconsistencies, if one of the two occurrences are forgotten.

3. Instead of copying the license text to each entry, only the identifier (e.g. MIT) has to be defined. The template appends the correct text automatically.

4. Defining a template and a model of this size actually made the handling more easy and future changes can be made much faster, while keeping the file consistent.

1 PHP: Hypertext Preprocessor, earlier: Personal Home Page
2 Cascading Style Sheets

In larger projects, the template can become very complex. Using intermediate models as described in section 2.5.3 that are generated from the source model, can reduce the complexity, branching and nesting, because the intermediate model has either the same or a correspondingly pleasant structure. Nevertheless, templates will get very big very fast and are bound to the structure of the target platform. In [Stab] this approach is called "Templates and Metamodel", while *Metamodel* is the intermediate model, that is used to generate code directly.

API-based Generators

The downside of templates are their untyped and unsafe character. It is possible to make small syntactic errors, that are unsafe and could cause the result to fail. Especially in interpreted target languages like PHP or HTML the error only occurs when the respective line is parsed or ran. Since the template elements are scattered in the file and important characters are hard to make out, this approach might be unsuitable for large projects.

An API-based generator is a programming library for a development language, which provides an interface to the AST[1]. It's purpose is to build up an AST using a type safe language like Java. Now the generator can be structured with Java's language features like Polymorphism, Interfaces and inheritance. The API makes sure, that only syntactically correct code is produced, because the AST does not allow it.

It is also possible to generate code for different target languages, which use the same AST. For example, it is possible to generate Kotlin instead of Java code, since Java is a subset of Kotlin. Also, the .NET runtime makes it possible to generate Visual Basic, C# and C++ code from one AST.

Listing 2.4: Using an API to generate Java code, written in Swift

```
1 let studentClass = ClassDeclaration(named: "Student");
2 studentClass.baseType = personClass
3 studentClass.typeAttributes = .public
```

Listing 2.5: Java code result of lising 2.4

```
1 public class Student extends Person {
2 }
```

Listing 2.4 shows an example usage of an AST-based Java code generator, which is written in Java itself. This form of meta programming allows the creation of any valid code by using an object oriented interface. Since it is independent from any textual

1 Abstract Syntax Tree

representation of the language, syntax errors in the generated code are impossible and any other language with an compatible AST can be generated.

The AST framework could make use of reflection to provide a more safe interface. For example, it could be possible to reference existing Java classes not by it's string name, but rather by using direct type references, like $typeof(System.Object)$. It might limit the set of compatible target languages, but in the example of Java, Kotlin is still compatible.

Using jQuery to manipulate or build HTML content is also a form of API-based code generators, since jQuery provides a safe[1] interface to the DOM[2]. The generated HTML code can then be read from the jQuery AST object.

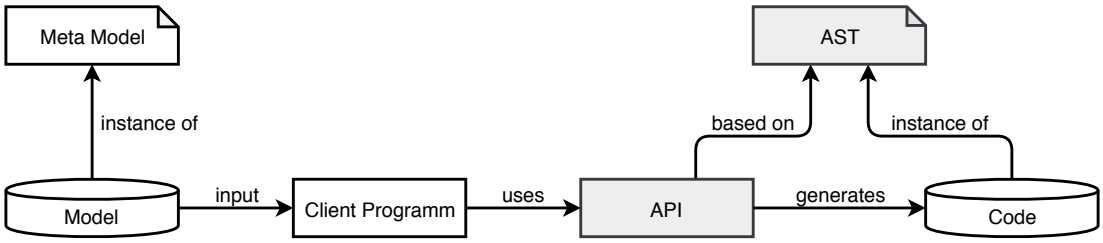

Figure 2.6: API based code generator infrastructure

Figure 2.6 illustrates the infrastructure of an API-based code generator. The darker rectangles (API and AST) represent components, that are independent to the domain, but are directly derived from the target platform. That means, that pre-existing solutions[3] can be used.

This approach is suitable for large projects, that need a lot of generated code. If most of the generated code is boilerplate code, which has only few injected lines, templates might be the better approach. Since templates represent the resulting structure, they might be easy to read. API generators are more maintainable since changes are type safe and less sensitive to errors. The initial expense could be higher in comparison to templates. It is possible to mix these approaches by generating parts independently.

1 Not all jQuery programming errors can be checked on compile time, despite a typed interface using TypeScript. Syntax errors in the generated code are not possible.
2 Document Object Model
3 E. g. CodeDOM for .NET Applications https://docs.microsoft.com/en-us/dotnet/framework/reflection-and-codedom/using-the-codedom

2.6 Model Interpreters

Instead of generating code, it is possible to run the model directly. The model is parsed by an interpreter, that uses the meta model to define an AST, which is the internal representation of a model. This representation can now be transformed to improve handling. It is possible to generate internal code out of the model which then is interpreted. Examples for that are Java or JavaScript, which are converted to byte code that is then ran in a virtual machine. The actual interpreter of the byte code is independent to the original AST, so a more modular infrastructure is possible. Another approach is to run directly on the internal data structure. This methods seems easier at first, but ties the run time directly to the AST which makes maintaining and introducing changes harder.

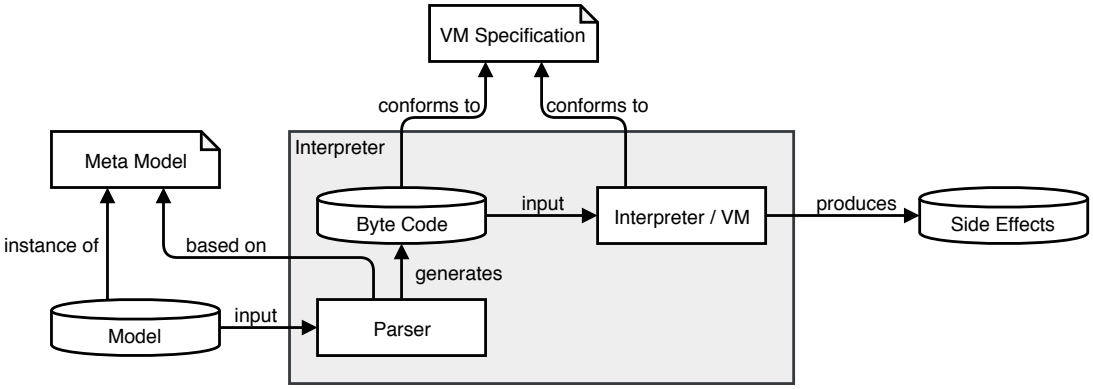

Figure 2.7: Interpreter infrastructure for common code interpreters

Figure 2.7 illustrates a typical interpreter that is used for *JavaScript* or *Python*. A model interpreter can use a similar approach, by transforming a model into a domain specific byte code or traversing the AST directly. To decouple the meta model from the interpreter, a intermediate data structure can be created by transforming the model. That way, a more modular infrastructure is achievable. Also, the transformed data structure can be optimized for speed by evaluating it's internal relations and constraints before it's is run. This will improve execution speed of repeating code like recursion or iteration. The approach of how to run the model is then directly dependent on the structure and therefore the domain. Also, the different approaches can each have specific levels of abstractions. The *V8 JavaScript* engine runs on a very low level of abstraction, since assembly-like byte code is interpreted. The interpreter that is discussed in future sections, mos, performs on a higher level, since it is executing user oriented tasks like sending HTTP[1] requests or displaying GUI[2] elements with data.

1 Hypertext Transfer Protocol
2 Graphical User Interface

An interpreter is dynamic and can swap or extend the current model. In the domain of web pages, JavaScript can be loaded on demand without recompilation. In the domain of mobile applications, the model can be changed or replaced without updating the app itself. Models only have to be compatible on the text syntax level.

3 Concept

This chapter will describe the goals of mos and it's iOS implementation. The first section will describe the informal requirements to the model. Also, the model's evolution and progression is discussed, which is relevant to the main idea behind the model interpreter.

3.1 Implementation Goals

The main problem of mobile app development is the iteration process, which consists of collecting requirements, creating conceptual prototypes and implementing on the two main platforms. Since requirements are informal, incomprehensive or change during the development, multiple iterations have to be done to converge to the final product.

To minimize implementation times and be able to do more iterations, *Same Day Delivery*[1] tries to eliminate the times, that is independent to the customer. That means together with the customer the app is designed and the model is created in one session. Since the functional scope is already defined and all of the feature are implemented in the interpreter, the domain expert who is directing the meeting can create the model live and on the fly. The App can be tested by the customer directly, without the need to compile an custom app. Common tasks like downloading data from a web source, forms and user interaction are configured on a high level, so the customer is able to understand aspects of the app.

3.1.1 Functional Model Requirements

To be able to validate the meta model and an implementation, requirements have to be defined. A concrete model also has requirements, which are application specific however. That's why, only the meta model and the implementation are considered. Since this chapter is about the concept of the mos interpreter, the discussion on the requirements of the implementation are found in chapter 4.

1 The term originates from parcel shipping and means that a parcel arrives at the customer on the day of the order.

The following paragraph lists the functional requirements to the meta model. Those are aspects that have significantly influenced design choices. All of the listed requirements are high level ones, that reflect the paradigm. However, the number of actual concrete requirements would be too high to be presented here.

1. **Separation into independent sub-models**

   An app loads a root node, which then can navigate to a different node, or show it's preview. Every node is theoretically independent and can be used as root node. To enable interaction between nodes, an action that displays a new node can inject additional data into the node. An example is a master-detail-view, which is a common pattern in application design. A master view is a list of all objects. It knows everything and is connected to the database. When the user clicks on one of the objects, the detail view is presented by passing either the database ID or the whole object as parameter. The same approach is possible with nodes, which results in the detail view node not being independent to the master view node.

2. **Fully customizable layouts**

   A layout can be defined by arranging different kinds of layers by configuring layout constraints. Also, native navigation bar, tab bars and side menus can be set independent to the layout. All of the common layer types are supported like text, image and color. Those layers have layer parameters that make them customizable. Position and size can be defined with constraints, by binding the edges of a layer to the edges of other layers or the superior container. The binding can be modified with additional margins so that dynamic and portable layouts can be defined. The system is inspired by Apple's Auto Layout[1].

3. **Declarative and reactive behavior**

   Everything that is depended on something else needs to update on changes. For example: If a layer is depended on an SQL query and the corresponding tables change their contents, the layer needs to update itself with the new data. Another example are layers that have a form data key and thereupon synchronize their contents with the view's form. Layers can also have a setting data key, which enables a layer to synchronize it's value with a field in the database. This makes it possible to use the value in SQL queries, apply it to templates or uploading data to a service.

1 Auto Layout calculates absolute positions and sizes based on constraints between layers. https://developer.apple.com/library/archive/documentation/UserExperience/Conceptual/AutolayoutPG/index.html

An example that enables a React[1]-like behavior is the *layout layer*. It's parameters are an SQL query, a layout name and a template name. The layer runs the query and applies the data to the template, and the template to the layout and displays the layout. If one of the tables that are selected in the query changes, the *layout layer* re-renders itself.

4. **Separation of data, templates and layouts**

   A template defines layer parameters by taking a dictionary of data and applies it to itself. The result can then be applied to a layout. A template overrides layer parameters. String literals inside the template can contain mustache[2] handles, which are evaluated against the data. This allows reuse components and enables dynamic decision making during run time. A layout-layer can dynamically load a sub-layout and apply a template. Since the name of the template is part of the layout-layer's parameters, it can be evaluated on run time and be modified by a mustache syntax.

5. **Database definitions**

   An SQL database can be designed, that is in sync with a data source. Fetch intervals, queries and structure can be defined. Column based queries for inserting data from a data source enables a structure, that is different from the source.

6. **Actions**

   An action can perform common tasks like uploading data, setting layer values, or triggering fetches. Actions can be fired by clicking on a layer or are triggered by other actions, which take the result of the previous action as context data to use. The action's parameters can be customized by using handlebar notation and injecting context specific data into the actions.

3.1.2 Non-Functional Model Requirements

The following paragraph lists the non-functional requirements.

1. **Easy to understand**

1 Reactive and functional UI framework. https://reactjs.org/
2 The string *{{data.field}}* is replaced with the corresponding value from an object. https://mustache.github.io/

Existing models are easy to understand. Names are clear and there is no room for interpretation. A model developer has to be able to understand an unknown app's structure and perform changes.

2. **No boilerplate code**

   All of the configuration contains highly concentrated information and no boiler plate code has to be defined. Similar configurations can be combined to one reusable resource[1].

3. **Easy to parse by an interpreter**

   The model is easy to parse and is encoded in a common format. Validators can check for simple errors. While parsing performance is not the issue, the interpreter's code base needs to be as simple as possible to avoid errors and to be maintainable.

4. **Backward and future compatibility**

   Adding new functionality has to be added without any old interpreters to fail on new models and a new interpreter to fail on old models. Also, every new feature as to be developed with the future in mind. The feature has to add important value to the models capabilities and has to fit into the design principles of the current feature set.

All of the above functional and non-functional requirements are developed both with a bottom-up and a top-down approach. Since the interpreter contains most of the business logic and implicitly defines the meta model, it is one of the key aspects. That is why, the meta model is structured to be parsed and interpreted in a simple and direct way. In the same way, the usability of the models are also considered, so that both sides are balanced.

3.2 Model Syntax

To meet the majority of the non-functional requirements, JSON[2] is used to define the model. It allows the client to parse common structures and define optional and default values in an maintainable way. Also, since the model uses declarative paradigms, C-like imperative language are not suitable.

1 Layouts, Templates and Nodes can be reused.
2 JavaScript Object Notation

The model is divided into sub-models. The first model is called root node and is loaded by the interpreter on the first app start. The URL to the root node is compiled into the app. A node defines some of the meta data, static data, a starting layout reference and events. The meta data contains an URL, called *Update URL*, that is used by the client to synchronize layouts, templates and database definitions. Database definitions also contain URLs that are requested if needed to synchronize actual content data.

Next to node definitions, template, layout and database configuration files are part of the model. The requesting the root node from a server will also append the additional files, that are cached by the client.

The client implementation is able to optimize the data usage by only reloading data that has changed. HTTP techniques like "etag"[1], "modified-since"[2] or delta updates[3] are used to optimize the update process. The model's job is to interconnect web services and the client. Optimizations can be implemented without changing the model, since they are not coded into the model in any way.

3.2.1 Drawbacks

JSON as a model syntax leaves a lot of room for interpretation, since the semantic and syntax are strictly separated. A DSL that is parsed by an compiler has semantics integrated into the compilation or parsing process, which allows finding errors much earlier. A common example is the parsing of a expression tree, which uses a top down parser, that parses parentheses. Depending on how the parser is constructed, it allows incorrect parentheses or rejects them because of a parser error. Defining the same expression as a JSON object

This issue can be fixed by using JSON Schema[4]. A schema is a JSON object which describes the syntax of given JSON data. It is possible to define required keys, possible values and other rules, that could help the model creator to find error faster. Also, a static context analysis could be used to catch even more errors. There are many occurrences, where layer, query and other objects are referenced. Those could be checked statically or auto-completed in the editor. At the moment, these tools are not implemented and typical errors occur regularly. The problem is, that those tools take a lot of time to develop, while they do not add any new functionality to the model or the interpreter.

1 Entity Tag, a unique identifier assigned by the web server to specify a resource and it's version.
2 A HTTP header containing a timestamp.
3 Requesting a resource by */api/file.60.json* will deliver the changes of the last 60 minutes.
4 https://json-schema.org/

The systems limits it's functionality and new features are hard to implement, since they have to be designed to work together with all of the existing features. This means, that the reason of why something is not possible to implement at the moment are incomprehensive and the customer cannot understand, why new features are much more expensive. An example would be custom user interface animations. While it is easy to hard-code some animations to specific elements in a classical app, it is much harder to define a representation for the model. That is because animations then have to be valid on any scenario. If an animation looks weird, there has to be a decision if the model is badly designed, or there is a error in the client code. Also, the level of abstraction has to be defined so all of the future animations can be expressed and the way of defining the animation must be compatible with the different platforms.

3.2.2 Iterative Development Process

The functionality of the model and it's interpreter is constantly evolving. Layers and actions can be extended easily and new features can be added really fast. The development method that is used in the example of mos is an agile iterative development approach. A new feature needs to be implemented into the platforms and a model has to be defined, that demonstrates it's aspects.

Most of the time, new features emerge because an app that is in development needs a feature, that is currently unavailable. The development of new functionality is avoided by first trying to implement the app with the existing syntax. For example, instead of developing highly specialized layers, the existing layers can be configured to build a new one. This approach is used by UI[1] frameworks to reuse existing code or by RISC[2] processors to reduce the chip's complexity.

The syntax is developed top-down and bottom-up. To be able to have a systematic approach to model-driven development, the simplicity of the syntax is one of the key non-functional requirements. On the other hand, a simple interpreter is also important, so some syntax elements are built to fit the current interpreter implementations. That is why, the system is developed by considering all levels and iterative development is used.

Figure 3.1 shows the informal development approach that is used to develop a model. The method is highly agile and team oriented, since multiple code bases have to be managed by different kind of people. Service-side services, the different platform interpreters and the model itself have to be developed in parallel.

1 User Interface
2 Reduced Instruction Set Computer

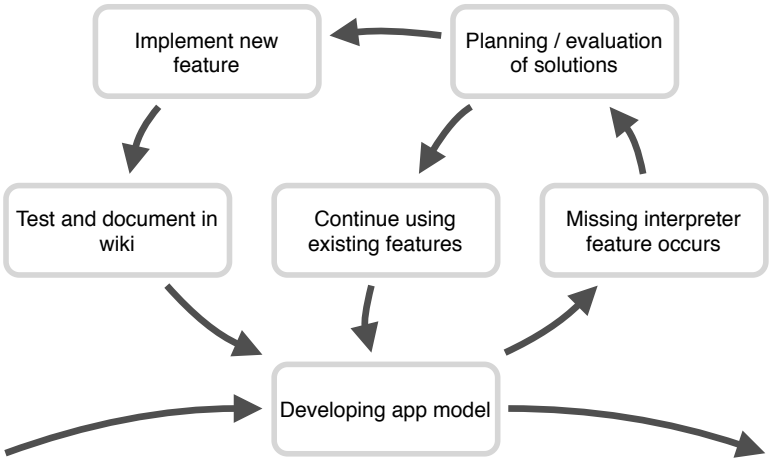

Figure 3.1: Iterative development

3.3 Client Application

the next section will outline a possible architecture for the client application. Since the main artifact is an iOS framework, from which only the main method has to be called, the bundle's architecture is independent to the framework's functionality.

In addition to the interpreter and a model, there are additional artifacts and software components, that are needed to support a mobile application software platform.

To build an instance of the interpreter app, additional compile time data has to be integrated. To automate this process, the compile time configuration is used to create an app from a template. Figure 3.2 shows the three states through which the development artifacts pass, until a usable app is generated. The following subsection will describe the components in detail.

3.3.1 App Configuration

Mobile applications for Android and iOS need to compile application specific information into the app. This meta data can not be loaded in run time, because the operating system needs them to manage the apps. Examples are the build number, version string, interface orientations and the root URL. This configuration is not fully platform independent, since the different systems require different information. In the case of iOS, the development tool Xcode, which is required to build iOS apps will not build an app until the needed meta data is provided.

The *App Config* also contains media like the app icon and the splash screen image. Also, the *Pre-Load Bundle* is included, which is a directory and contains images and nodes. This directory is used to load data from disk instead of the network.

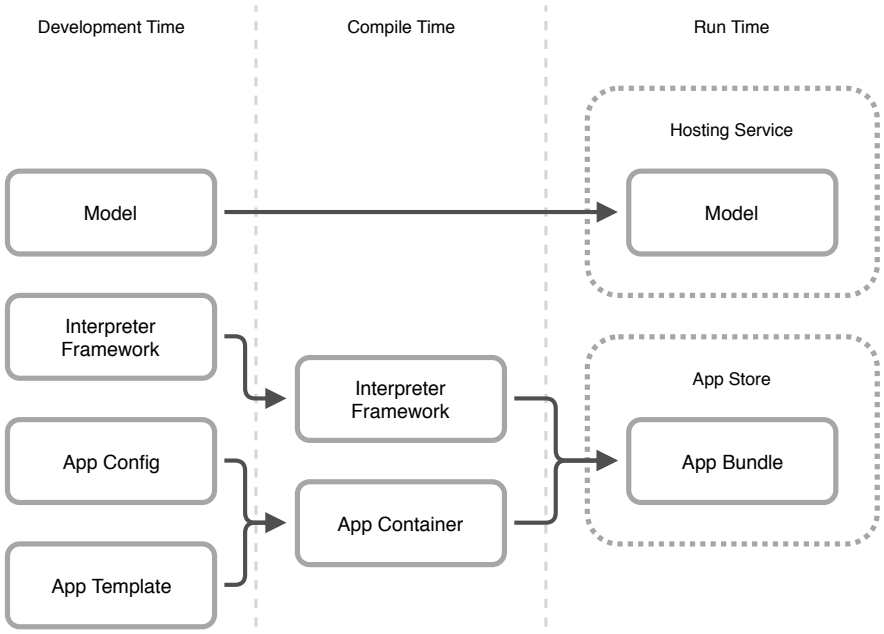

Figure 3.2: Compilation of development artifacts for mos

3.3.2 App Template

The *App Template* is a template file structure that is used to create a run-able app. On its own, the *App Template* is an incomplete app container that contains handlebars placeholder. A grunt task takes the *App Config* and replaces the placeholders. This way, an app container is created, which is able to compile and ready to be submitted to the store. A continuous integration environment should be able to build all apps, given a list of all app names.

3.3.3 App Bundle

The app bundle is a full-fledged application, that is an instance of a specific app model. It is the result of compiling the *App Config*, the *App Template* and the *Interpreter Framework*. It does not contain the model itself, because it is loaded from the web the first time the user opens the app. This way, the model can be changed on the fly, while the app itself is still waiting for review or ready to release.

The model is interpreted by the Bundle, which lets the model run inside a virtual environment. The mos run time enables declarative model design and handles the data flow. Similar to a virtual machine, the model can access subsystems and interfaces that are normally not available in the target platform. For example, a custom event system handles app wide events and actions. Also it provides the interface to the custom layers. Those are layers in the view hierarchy that have more functionality then the default ones.

The biggest aspect is the calculation of changes. Since relations are defined declarative, changes to the data have to update and re-layout the layer accordingly if needed. Also the update of database data and the scheme is a big part of the system.

3.3.4 Functional Interpreter Requirements

The following section will list the functional requirements of the interpreter. The functionality of the platform that is described by the model is omitted due to its comprehensiveness. Instead other requirements will be listed, that reflect some of the design choices of the platform. The behaviors, that are discussed per requirement are not expressed by the model, but included in the interpreter.

1. **Reflects changes to the model**

   After the configuration changes, the interpreter app loads the changes and updates it's behavior. This includes database configurations, layouts and caches.

2. **Offline capable**

   All of the content that has been displayed in the app must be cached and available for offline use. This includes nodes, images, layouts and templates.

3. **Cache-first policy**

   All nodes are loaded from cache. If a node is not available, the node will be downloaded from the network. If the not is available but is expired, the new version will be downloaded. Until the download finishes, the old node will be displayed.

4. **Clear cache and data**

   The user must be able to clear the cache. This way, some troubleshooting and customer support is possible. It may happen that a faulty node is cached that is not compatible with the other nodes, or is not configured with a correct cache lifetime. To prevent the need of re-install, a clear cache action is available that the user can execute himself.

5. **Poor configurations lead to fall-backs**

   Dynamic model errors must lead to safe fall-back behaviors. Under no circumstances should the app crash or display incomprehensible error messages. Since there is no static model analysis, the app must be prepared with default values.

Some of the special behavior of the interpreter is available to the model to be described. The caching policies of a node can be configured in detail. Also, the cache can be invalidated on the basis of a regular expression. Those are a few examples of the model and the interpreter

3.3.5 Non-Functional Interpreter Requirements

The following section will list non-functional requirements of the interpreter.

1. **Low battery usage**

   Using the app must feel as lightweight as possible. That is why, the battery usage must be kept low. Since all of the modern mobile operating system have battery management tool integrated into the system, users can see which apps use a lot of battery. If the app is listed there, the probability is high that the user will consider deleting it.

2. **Fast app start**

   The must start as fast as possible. Especially the first start is important, because it's the first impression that a user gets from the app. Because of a strict cache policy, other starts are faster anyway.

3. **High UI and animation performance**

   The UI should always be responsive and adequate loading indicators must be used. It is accepted, that poorly designed models can lead to slow UI. This is possible because of the high level of detail of the models.

4. **Low network usage**

   The app should not load redundant data or load the same data multiple times. Client and server-side strategies can be used.

5. **Small download size**

   The size of the app at the time the user downloads it from the app store must be as small as possible.

6. **Small app size on disk**

   The app should not accumulate to much data, while in use. New cache entries must always override old ones.

3.3.6 Implementation Weak Points

The given interpreter client has a number of technical limitations and inconsistencies. The following subsection will outline some of the major issues, that are part of the re-engineering process. Most of them are non-functional requirements, that are met while the app was in development and the feature set was smaller. While the app grew, conditions and the correlations between components became more complex. There have been two main problems, that can be subject to a major refactorization.

1. **UI elements are re-painted to often.**

   Every change to a layer results in a re-layout and a full repaint of the parent container. If the layout frame did not change, a re-paint is not needed. Complex layouts like search results in combination with per entry conditions result in a slow UI with up to two seconds wait time between rendering frames.

2. **Every operation runs on the main thread.**

   To prevent race conditions and visual noise, the entire application run only on the main thread. Complex layouting and database-driven UI elements block the UI thread. On fast devices, this works well, because there no visual noise. That means that the user doesn't see how the UI is getting built, but instead sees the full UI after one display refresh frame.

3. **No abstraction between native platform code and independent mos behavior.**

   The app uses native interfaces like the view stack to organize it's contents. Instead of using a virtual node stack, the view hierarchy is traversed to find nodes. Also, every mos element manages or has a reference to the native result.

3.4 Native Approach

One re-engineering approach is to fully combine native elements with mos behavior. The main reason for this approach is the simplicity of the code and the low distance to the native components. This approach works for a rather small interpreter feature set. To compensate bad performance, simple and highly optimized code should be used. The code is easy to understand and simple optimizations can be applied.

The idea is, that native components are extended with mos behavior. That means, that everything happens directly on the main thread and on native UI elements.

One key element is a centralized change management system, which tries to collect all changes that happen within one user interaction. The changes are then combined to reflect one resulting change. The problem is, that applying changes to the native elements can result in new changes. If a layer is configured to change it's state based on the child count of another layer, the UI has to be rendered multiple times, which is very expensive.

While the feature set grew, the requirements to the system got higher. New features like the application of changes to layers in a different node result in a high workload on the main thread. Introducing concurrency now is only possible on a few marked-out aspects, because the code operates on the UI in an unstructured way. UI operations are only allowed on the main thread, otherwise, the change will be ignored. The opposite of this approach is explained in the following section, which tries to separate mos and UI code.

3.5 Virtual Machine Approach

Popek and Goldberg [Pop] defined 1974, that a virtual machine is a duplicate of a real computer. But as programming languages evolve and virtualization allows more complex systems, a virtual machine does not need to have real correspondence, but can rather implement any runtime system. Popular virtual machines, that are not based on real computers are the JVM[1] or the V8 JavaScript engine.

As described in section 3.3.6, the current iOS implementation is hard to maintain and has performance issues, because native code and mos behavior are mixed. There are only few specific conditions under which performance optimizations by concurrency are possible.

To improve performance and maintainability of a model-driven mobile application, the approach of virtual machines could be used. The VM[2] is a software system that is implementing the platform specific behavior. In the context of mos that would be conditions, states, layouting, data and caching. It could be integrated as an external framework and is independent of the actual platform and user interface. The system would run in a own thread and not on the main thread.

1 Java Virtual Machine
2 Virtual Machine

3.5.1 Native Components

Every interface to the native APIs are managed by custom, platform depended components, that can be registered in the VM. For example, an SQL interface is provided that will open connections and run SQL statements. The UI component will be asked to render some layers. All the layers are already lay-outed and their respective sub-layers evaluated. It should be as easy for the native components to perform their work, since they only translate the system's objects and tasks to native ones. The logic between elements will be handled by the virtual machine.

Since the system will run on it's own thread, the UI will be fast and responsive and only UI related work will block the main thread and with that the UI.

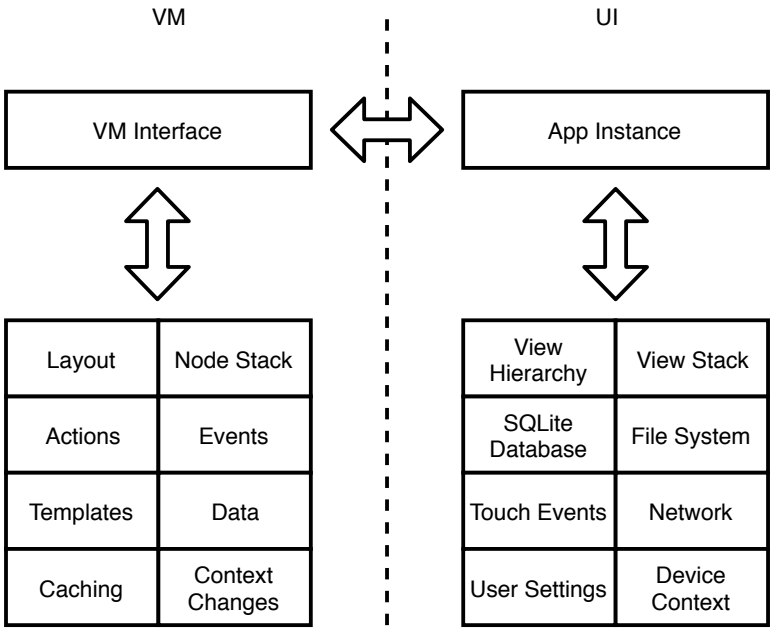

Figure 3.3: Virtual machine, to separate mos from platform code

Figure 3.3 outlines the basic concept of the architecture. The main challenge is the interface between the VM and the native app instance. This interface must be efficient and thin, but also implement all the changes that can occur to the UI. The implementation of the VM logic is easier, since most parts are already implemented, but have to be extracted and refactored to not have circular dependencies. Existing performance improvements should be evaluated if it suites the new system. One example is the custom ContextChange pattern, that will collection all changes and will merge them to one resulting change that will be delegated to the layers. This will be used if multiple actions will be fired. If the database is changed several times during this time, there will only be one notifications to depended layers. This should be used in the new VM system to prevent multiple redundant messages to the host.

3.5.2 Virtual View Hierarchy

One of the major benefits is the virtual view hierarchy. Inspired by React's virtual DOM, the VM handles a virtual view hierarchy, which is used to calculate layouts and the layer changes. While it is very expensive to layout and render native view components, virtual elements can be lay-outed very fast, because no user interface render system is involved. This means that all changes and their resulting changes can be applied on the virtual view hierarchy in multiple iterations without the need to interfere with the main thread. Layers that have changed will be marked as dirty and after every change is processed, the actual UI is updated to reflect the virtual hierarchy.

To minimize the amount of change application to the view, only on useful occasion The application of changes in the native view should only happen on useful occasions, to minimize the main thread utilization. The web UI framework Angular uses a change detection system, that hooks into the browser specific event system to trigger an update only if changes are even possible.

Table 3.1 lists some events that can result in DOM changes. Angular hooks into those events by using a low level patch mechanism[1] and redirects them to the actual angular application, which can be considered *user code*. After the application is done with it's calculations, the DOM updated be re-evaluating the HTML templates on the basis of the new values.

This idea can be transferred to the mos system as shown in Table 3.1. Network request action in mos are synchronous by definition and there are only a few event's, that can be triggered by the user. This reduces the complexity and thus the probability of errors.

Category	Examples	mos equivalent
DOM Event	click keyup onreadystatechange ...	Layer Actions App Events
Timer Event	setTimeout setInterval ...	setTimeout Action
Network Response	Ajax WebSockets ...	-

Table 3.1: Events, that are used for change detection by Angular and their mos counterpart.

1 Zone.js https://github.com/angular/zone.js/blob/master/STANDARD-APIS.md

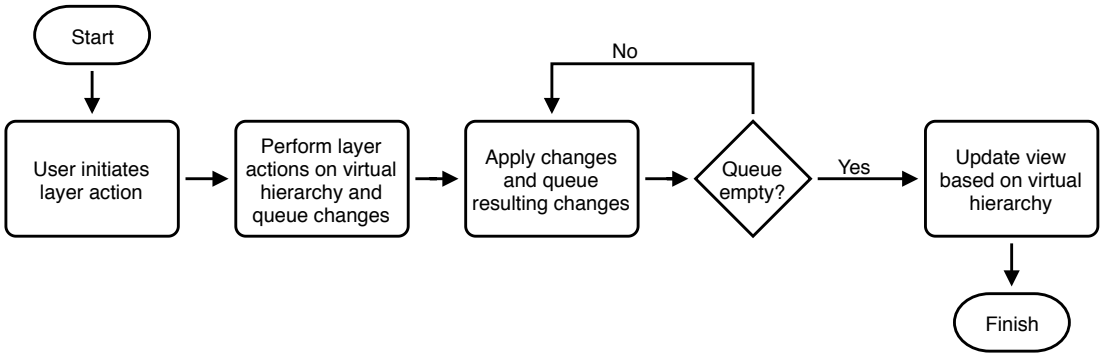

Figure 3.4: Change management cycle

Figure 3.4 outlines the processing of user interaction. Notice the loop which operates on the changes queue. Bad configuration could lead to an infinite loop. The old single threaded approach which operates directly on the native UI has to apply the changes to the UI on each iteration, because only that way, the new resulting changes are created. This means that the application is not responsive for the time the new view is evaluated.

The biggest challenge is the application of changes to the UI in the most efficient way. A possible solution could be a unified interface that can update the frame, the layer value and the sublayer list.

3.5.3 Shared Code

It is possible to develop the VM system as an external framework. If it would be written in a language that can be compiled to native frameworks like KotlinNative[1], this code could be shared between iOS and Android.

The main goal is, that the virtual machine can run on any mobile operating system, implements and represents the mos system and it's design philosophies. A native client should provide platform-specific implementations like the UI and the device context.

3.5.4 Testability

With the current implementation, only some isolated parts can be unit tested. Critical parts like layouts, conditions and action cannot be tested, since they all rely on the native UI components. Also, the code has many circular dependencies, which makes

1 KotlinNative builds native C++ libraries and Objective-C frameworks from Kotlin code. https://kotlinlang.org/docs/reference/native-overview.html

mocking impossible. Behavior of mos is directly depended on native UI elements and vice versa.

Automated UI tests are expensive to maintain and are hard to evaluate. Using a virtual machine that runs headless[1], unit tests are possible since any app can be used to test against and simulate user input. It is possible to inject virtual app models into the VM that can be used to test against. This allows for fast test setups and evaluation.

1 Headless is a mode of virtual machine hosts, that runs the guest system without a user interface.

4 Implementation

In this chapter, the approaches that are described in chapter 3 are demonstrated using code examples and the real reference implementation. In the first section, the initial state of the interpreter is explained. This is the starting point from which the re-engineering options are applied.

In the next section, the first approach is examined. This is the option that is described in section 3.4 and has been actively developed.

The last section will outline a possible virtual machine implementation exemplary, that was described in section 3.5. Since the virtual machine approach has not been implemented yet, a simple demonstrator has been design, that implements the key ideas.

4.1 Initial State

The original iOS interpreter is written in Objective-C and uses *AsyncDisplayKit*[1] as an asynchronous UI framework. It allows instantiation and lay-outing of view components on any thread. This means, that load intensive tasks can be swapped to background threads.

This makes synchronization of mos specific behavior like condition evaluation and state management harder. Also, custom view implementation that are used very often are more challenging to build, because they have to be a subclass of *ASDisplayNode*, which represents one simple view component. In addition, the framework's future was unknown, since it has not been maintained and was licensed by Facebook. It has no native Swift API. Later, the framework moved to Pinterest, where it has been maintained more regular.

Because of the high dependence to *AsyncDisplayKit* and the difficult maintenance, the next iteration of mos, which should use Swift, needs to move away from the UI framework.

1 AsyncDisplayKit was renamed to Texture. https://github.com/texturegroup/texture/

The code basis that was used to utilize the framework became obsolete because native views, that are enriched with mos behavior, should be used.

4.2 Native Approach Implementation

Instead of *AsyncDisplayKit*, that is described in 4.1, a approach should be used, that primarily uses the native UI and the main thread. The main approaches will be outlined and explained in the following sub sections. This approach is fully implemented and is used for the actual applications that are described in section 2.1.

4.2.1 Native User Interface

Every layer operates directly on their native representation. That means that if changes need to be applied, the layer manipulates it's UI component directly. This is necessary because some of the layouting calculations are based on native UI APIs.

An example would be the size of text in a label layer. A text layer can be configured to update it's size automatically based on the text content. Also, a maximum line count could be defined. Since the native text API only supports the calculation of the text size based on a constrained size, but not based on a line count, another approach must be taken. In this case, the text and the calculated text size is applied to the label. After that, *sizeToFit()* is called on the layer. It's a native method, that shrinks the label based on the maximum line count. The resulting size is then used as the layer's frame to layout the remaining layers.

Another example is the size of all sub view of a grid layer. Since a native grid layouts views based on a collection layout definition, the resulting size can only be determined by asking the grid on the main thread.

This means, that lay-outing and conditions happen directly on the UI. It keeps the code client and simple configurations are run fast. The following listing outlines a layer and it's behavior.

Listing 4.1: Layer class (excerpt)

```
1 class Layer {
2   let baseLayerParameters: BaseLayerParameters
3   var state: LayerState
4   weak var superlayer: SublayeredLayer?
5
6   init(baseLayerParameters: BaseLayerParameters)
7   func performActions()
```

```
8   func applyConditions(afterContextChanges contextChanges: [ContextChange
        ] = []) -> [ContextChange]
9   func apply(databaseChanges: [DatabaseChange]) -> [ContextChange]
10  func setNeedsLayout()
11  func size(constrainedBy constrainedSize: CGSize) -> CGSize?
12  func activate(state: LayerState) -> LayerStateChangeSeverity?
13  func view() -> UIView
14  func update(view: UIView)
15 }
```

Listing 4.1 outlines an excerpt of the Layer class. The function *update(view:)* is the bottleneck with the most significant impact. It's responsible for re-layouting it's native components.

4.2.2 Concurrency

Since changes are happening directly on the UI elements, the main work needs to be done on the main thread. This also means, that if a layer needs to re-layout, the superlayer needs to relayout all of it's sublayers. That is because of layout constraints, that are tied to the content might can change every layers frame. Another high workload is the update of sublayers based on a database query. This feature is called *InsertLayer* and it uses a layout definition and a database query, to dynamically create sublayers. Since layers can only be instantiated in the main thread, and changes have always be applied to the native view elements, the main thread is blocked for the time. It is possible to extract certain tasks to a background thread, but this would not follow any pattern and would lead to poorly comprehensible code.

4.2.3 Testability

Only a minor part of the implementation is testable, because most logic code is directly intertwined with UI code. Also, code depends on a lot of global references of app components like the current view or the database controller. This makes dependency injection impossible. Code that has been develop recently has been made testable by using injection patterns, which help to write white box tests. An example are the tree conditions of a layer state. If the condition tree is evaluated to be true, a defined state is applied. The condition tree is a recursive declaration of conditions, that can be combined this standard bool operators. The leafs of the tree reference various app context values like user settings or form data. Those values can be injected to make the evaluation of the tree testable.

4.3 Virtual Machine Implementation

This section illustrates a possible solution and alternative to the approach of section 4.2. To demonstrate the implementation, a small demonstrator has been developed, that will show the an example scenario. There is no actual implementation of this approach.

4.3.1 Base Elements

The first level of abstraction consists of an UI and a VM framework. The UI framework uses the VM framework.

The VM framework is an implementation of the mos logic like nodes and layers. It is independent to the native UI and device specific APIs. This example is written in Swift, so the idea is, that the framework can be used on every Swift-enabled platform like macOS and iOS. It is also possible to abstract the framework even further to allow it to run on Ubuntu. But while macOS and iOS share a lot of useful frameworks like CoreGraphics, Swift for Ubuntu only comes with the most basic types. The best solution is a SPM[1] framework, that builds also on Ubuntu. This way, it is ensured that there is no UI or platform code hidden in the framework. Most non-SPM frameworks are the ones that use platform frameworks like UI and are only part of the UI framework anyway. After a short evaluation, most current dependencies that would be part of the VM framework are SPM compatible. However, some of them are unable to build on Ubuntu, because they export an Objective-C API, which is not available under Ubuntu.

As shown in figure 4.1, the UI framework uses the VM frameworks and is it's delegate. The VM asks to display layers and to push new views. The UI then creates native views and provides the actual app and it's user interface. Also, the UI triggers VM events like touch of layer or app life cycle events.

Delegate is an software pattern, where frameworks store a weak reference of an object. This object implements the framework's delegate protocol and handles events that come back from the framework. It's similar to the callback pattern on asynchronous functions.

The virtual machine now consists of the public interface and it's internal logic. The public interface must be as thin as possible, so that most of the work can occur on the virtual machine's thread. For the UI to be able to display native views, the internal layer implementations also define a small public interface. It is used to provide relevant information about the layer. The UI delegate will be asked to display a certain pre-rendered layout, which consists of layers. Rendered in this context means, that all

1 Swift Package Manager

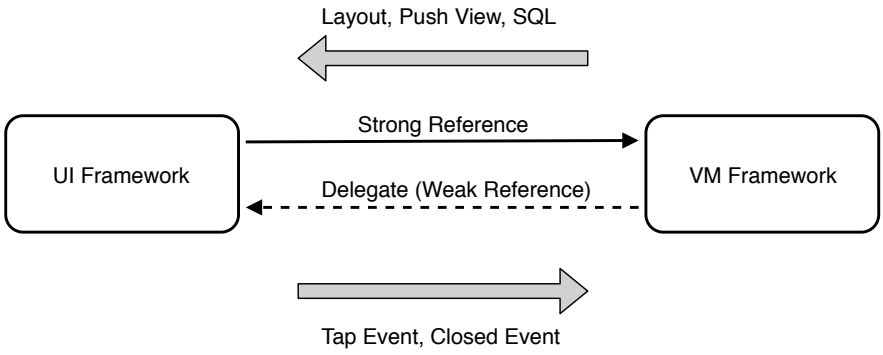

Figure 4.1: Basic interaction between VM and UI

the work of building the full hierarchy is done. This includes layouting and evaluating *GalleryLayers*, *GridLayers* and *InsertLayers*. The UI is now responsible for initializing the needed UI elements. This means, that for every Layer, there is a corresponding LayerView, that is provided by the UI framework.

This separation of layer and view has additional positive side effects. One example is the GalleryLayer, which displays it's sublayers as carousel with horizontal scrolling. It can be configured to enable infinite scrolling, which means that the gallery does not end at the last layer, but continues scrolling to the first layer. Since the scroll animation is continuous, the only solution has been to duplicate the sublayers internally and place them at the end of the gallery. If the user now scrolls across the boundaries, the gallery offset is moved back by its original layer count.

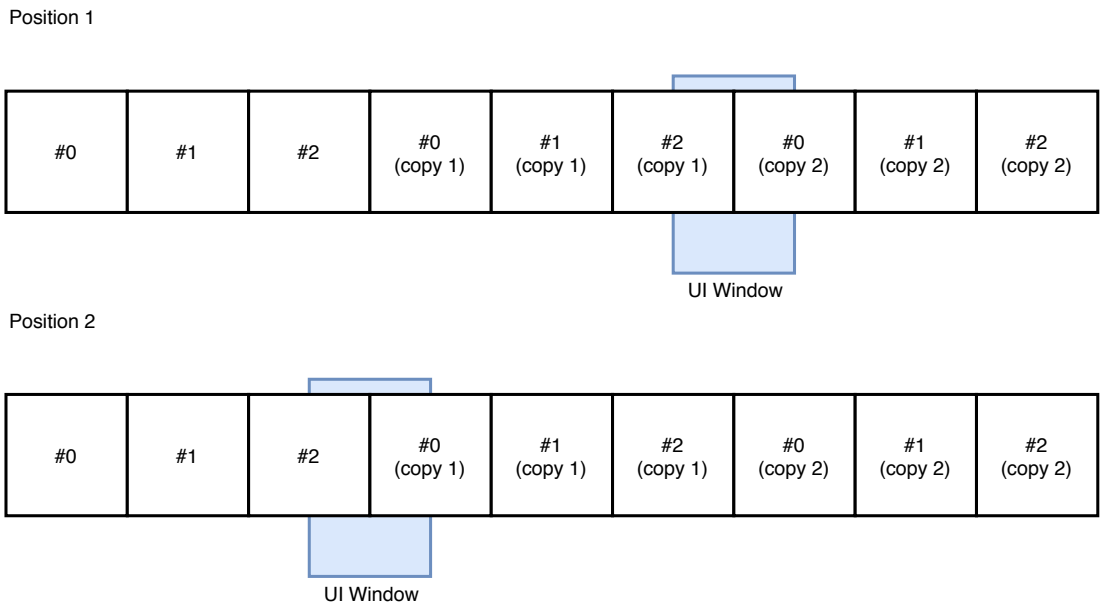

Figure 4.2: Simulation of infinite scrolling by resetting the window at the boundaries.

Figure 4.2 demonstrates this behavior. The initial state is that the UI windows is positioned at the first copy of the first layer *#0 (copy 1)*. If the user now scrolls past the end and is already seeing the second copies (position 1), the window is instantly moved back by exactly the width of the original layers (position 2). This way, the user cannot tell if the offset was changed and it feels like the gallery is able to scroll infinitely.

It is possible to achieve this with only one copy, but the iOS scroll view component will decelerate the scroll animation to fast if is realizes, that there are now more views to come. To have more buffer and therefore a soft user interface, two copies have to be made, which results in the layers existing three times.

The problem now is, that because of the duplication of layers, it is undefined which layer responds to actions that reference the layer by its name. The virtual machine approach solves this problem, because the duplication of the layers will be done by the view. This way, there is no layer duplication and the changes of the separate layers will be flushed to all of the native observers.

4.3.2 Change Management

The most important key element of this alternative approach is the concept of Dirt and Flushing. Dirt represents changes that happen to a layer a will need to be applied to the native view. Every Layer stores a set, which is filled with dirt instances. This makes a layer *dirty*. On well defined occasions that are explained later, a flush is executed. A flush will take the dirt of every layer and send it to the native views. The native view will then apply the changes that are represented by the dirt. The dirt set will then be set to empty.

Two important conditions must be met for this concept to be useful. The storage of the dirt must be a set, so that duplicates are not possible. This ensures that changes are not applied multiple times.

The second condition is the frequency of which a flush is executed. Similar to the dirt management of Angular and Zone.js[1], changes can only occur on special, well defined occasions.

Table 4.1 lists all the interactions that can produce dirt. Everything that happens in the VM originates from one of those interactions, so only after those, one flush must be executed. That means, that the most complex action configurations that change a lot of things results in only one flush so that the work on the main thread is as minimal as

1 See section 3.5.2.

Interaction	Examples
Layer Actions	Tap ChatLayer send BarcodeLayer onSuccess, onError SmartList Item Actions & Tap MapList Item Actions Autocompleter Cancel Actions TextBoxLayer Return Actions WebLayer navigation to af:// URL MapLayer Annotation Actions PickerLayer picked GalleryLayer Page Index Change
App Events	didStart didBecomeActive didEnterBackground
Update Manager Changes	Database Update
Timer Actions	setTimeout
Background Location Actions	startLocationTrackingAction (background: true)
Push Notification Actions	Payload action
Beacon Actions	enterBeacon exitBeacon
NFC Actions	onTagActions

Table 4.1: Interactions, that produce dirt, that needs to be flushed.

possible. Non-Layer changes like push and pop of views are not part of the dirt system. Instead, they are send after the flush, af special app events. This optimizes main thread performance, because only fully evaluated layouts should be send to the UI.

A flush is executed recursive, starting the root layer. This ensures, that layers, that are not part of the hierarchy anymore are not triggered, so that the UI is not performing work on layers, that will be removed anyway.

Listing 4.2: Key data structures of dirt and flush

```
enum LayerDirt {
  case frame
}

enum TextLayerDirt {
  case value
}

public class Layer {
  var layerDirt: Set<LayerDirt>

  public var frame: CGRect
  public var onFrameChanged: ((Layer) -> Void)?
```

```
14
15    func updateFrame(to newFrame: CGRect) {
16      self.frame = newFrame
17      layerDirt.insert(.frame)
18    }
19
20    func flushDirt() -> Void {
21      layerDirt = layerDirt.filter {
22        switch $0 {
23        case .frame:
24          onFrameChanged?(self)
25          return false
26        }
27      }
28    }
29  }
30
31  public class TextLayer: Layer {
32    var textLayerDirt: Set<TextLayerDirt>
33
34    public var value: String
35    public var onValueChanged: ((TextLayer) -> Void)?
36
37    func updateValue(to newValue: String) {
38      self.value = newValue
39      textLayerDirt.insert(.value)
40    }
41
42    override func flushDirt() {
43      super.flushDirt()
44      textLayerDirt = textLayerDirt.filter {
45        switch $0 {
46        case .value:
47          onValueChanged?(self)
48          return false
49        }
50      }
51    }
52  }
```

Listing 4.2 shows an excerpt of a demo implementation. The layers each have a set of their own relevant changes (= dirt). Notice that a developer must not forget to add the dirt to the set in case something changed. The functions *updateFrame* on the Layer and *updateValue* on the TextLayer make sure to add the dirt.

A native layer view registers itself for changes using the *onFrameChanged* or *onValueChanged*. So if the VM system decides that it is time to flush, the layers *flushDirt* functions are called. The TextLayer makes sure to flush it's own and it's superclass's

changes. The challenge is, to not forget to add dirt to the set and to represent the change correctly, so no change gets lost.

The same mechanism can be used to handle internal changes as well. To make sure, that layers who update based on changed database tables do not update multiple time unnecessarily, an internal dirt flag for every database table can be used. This way, an interaction first triggers the internal evaluation mechanism, then the internal dirt flush and the external dirt flush after that. Examples for layers that depend on a database query are the InsertLayer, GalleryLayer, SmartListLayer and the LayoutLayer.

4.3.3 Concurrency

Since the VM and the UI are separate systems on separate threads, they have to be synchronized. This must be done in a way, that the UI is not blocked, but events and changes occur in serial. A possible collision are the public fields of the layers. Those are used by the UI to initialize and change it's views. If new changes occur right after a flush, while the UI is still applying changes, the public fields could be written and read at the same time.

A possible solution for this is a additional state object, that is only written to right before the flush. This makes it possible to use observables, so no custom delegates or listener mechanism has to be developed. To implement this, *ReactiveSwift*[1] and it's observable property feature can be used. So instead of having a property and it's change handler in addition to the layers actual internal property, there are only the internal and the public one. A flush means, that for every dirt flag, the corresponding observable property's value is overwritten with the internal one. A major advantages are the operators, that enable structured observation and mapping of values. It also allows filtering of equal values via *skipRepeats*, to improve UI performance even more. The *skipRepeats* operator cannot replace the dirt flag system, but extend it. The dirt system can relieve pressure on the *skipRepeats* operator, because checking for equality on the sub-layer list for example might take very long, so it is better to mark the sub-layers as dirty if re-calculated so it can be overwritten only if actually changed.

The core idea to handle changes is that the UI calls the VM's APIs using *fire and forget*[2]. But the VM uses synchronization to wait until the main thread is done flushing. This way, the VM is always blocked while the UI is working.

1 https://github.com/ReactiveCocoa/ReactiveSwift
2 A call to the other system that returns immediately.

As a result, the UI is always responsive, except for the time it is applying changes. Also, changes are applied in the correct order, and always after one another.

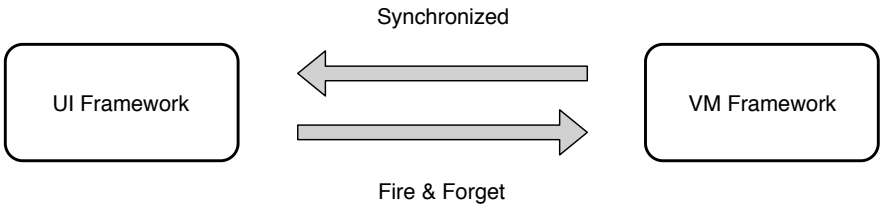

Figure 4.3: Synchronization of VM and UI

4.3.4 Proof of Concept

A proof of concept implementation has been developed, that represents all the ideas that are described above. It consists of an *AppHost*, which is a common iOS app bundle, but instead of launching an app, it calls the main method of *mosUI*, which represents the actual app. The *AppHost* is separated from the app executable, because it will be used to inject the custom app configuration like app name, icon and runtime privileges. *mosUI* is independent to the particular app instance and so it is lifted into an own framework bundle. It uses *mosVM*, which is the abstract implementation of mos behavior.

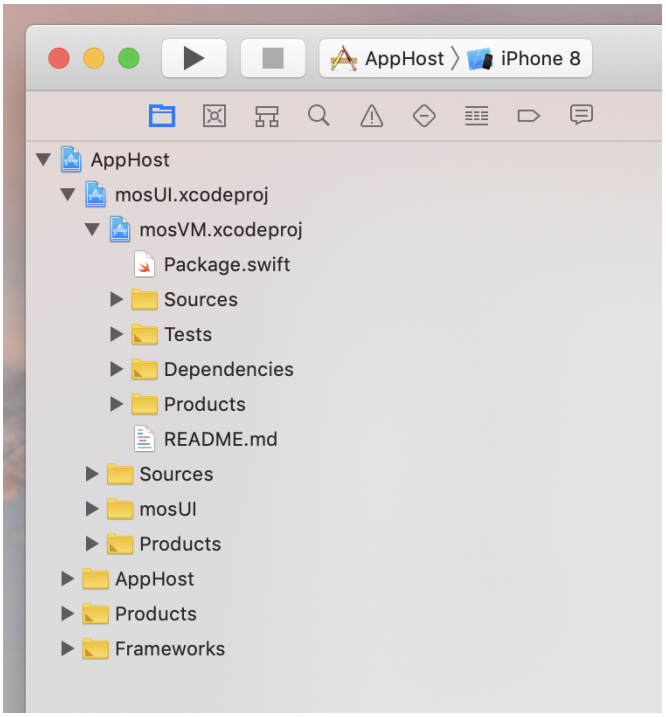

Figure 4.4: Project structure of a proof of concept implementation

Figure 4.4 outlines the project structure of the proof of concept implementation. Notice that *mosVM* contains a Package.swift file, which indicates that it is a platform independent swift package, that can be build for any Apple platform.

Implemented Features

The virtual machine is designed to support 2 layers. A TextLayer which has a value and a ColorLayer, which has no properties. Both Layer are inherited from a base Layer, that can be configured with a name, a layout frame, a form-data key and a list of actions.

A form is implemented, which are used by the layers to synchronize their values based on the form-data key. This means, that the layers share their value with layers that have the same form-data key. This logic is implemented in the virtual machine.

Each layer can have a list of actions, that are fired if the user taps on the layer. Currently there are two actions implemented. The GrowAction let's a layer's frame grow to the right. The ToggleTextAction will toggle the value of a layer between "lorem ipsum" and "hallo welt".

As described earlier in section 4.3.2, the dirt and flush mechanisms are implemented. The public API of the layers make sure, that if the user taps on the representing view, the notified layer will call the virtual machine to perform the actions. After that, all layers are asked to flush their changes which will then be applied to the view.

A centralized app instance object, which will always be passed to layers and actions holds all of the relevant components of a virtual machine instance. In this implementation, the instance object has the form object, the dispatch queue and the current layer list, that are currently displayed. A dispatch queue is an abstraction of a thread, which is used to run all the machine logic.

Example Configuration

To demonstrate the approach, a demo app is configured, by letting the virtual machine create layers on it's own, which are then displayed. Three layers are configured. A color layer with a size of 100 by 100 pixels, a TextLayer that displays the text "Click me multiple times" and another TextLayer that shows the text "... or me". Both text layers are configured to have the same form data key, so both layers should display the same value. In mos it is undefined, which layer "wins", but in this implementation, the first one sets the form value and the second has to adopt. The first TextLayer is configured with three ToggleText actions and one GrowAction. The GrowAction is configured with a the layer name of the ColorLayer. The GrowAction is implemented to find the layer

who's frame will be extended by a layer name. The ToggleText action is configured three time to simulate multiple changes. The result should be the same as if the action would only be fired one time. The hard-coded configuration is depicted in listing 4.3.

Listing 4.3: Demo configuration

```
1 let colorLayer = ColorLayer(
2   instance: self.instance,
3   layerParameters: LayerParameters(
4     name: "color1",
5     formDataKey: nil,
6     frame: CGRect(x: 10, y: 10, width: 100, height: 100),
7     actions: [ToggleTextAction()]
8   ),
9   colorLayerParameters: ColorLayerParameters()
10 )
11 let textLayer1 = TextLayer(
12   instance: self.instance,
13   layerParameters: LayerParameters(
14     name: "text1",
15     formDataKey: "value",
16     frame: CGRect(x: 10, y: 120, width: 100, height: 100),
17     actions: [ToggleTextAction(), ToggleTextAction(), ToggleTextAction(),
           GrowAction(layer: "color1")]
18   ),
19   textLayerParameters: TextLayerParameters(
20     value: "Click me multiple times"
21   )
22 )
23 let textLayer2 = TextLayer(
24   instance: self.instance,
25   layerParameters: LayerParameters(
26     name: "text2",
27     formDataKey: "value",
28     frame: CGRect(x: 10, y: 240, width: 100, height: 100),
29     actions: [ToggleTextAction(), ToggleTextAction(), ToggleTextAction()]
30   ),
31   textLayerParameters: TextLayerParameters(
32     value: "... or me"
33   )
34 )
```

Figure 4.5 shows, how the app behaves, after a user touches the first label, which is marked with a pink circle. The three ToggleText action are executed, which results in the first layer to changes it's value. It will notify the form, that his value changed so the second layer will apply the value too, because it has the same form data key. Then the Grow action is executed, which results in the ColorLayer to increase it's layout frame. All this was executed inside the virtual machine on a virtual view stack. Now that a

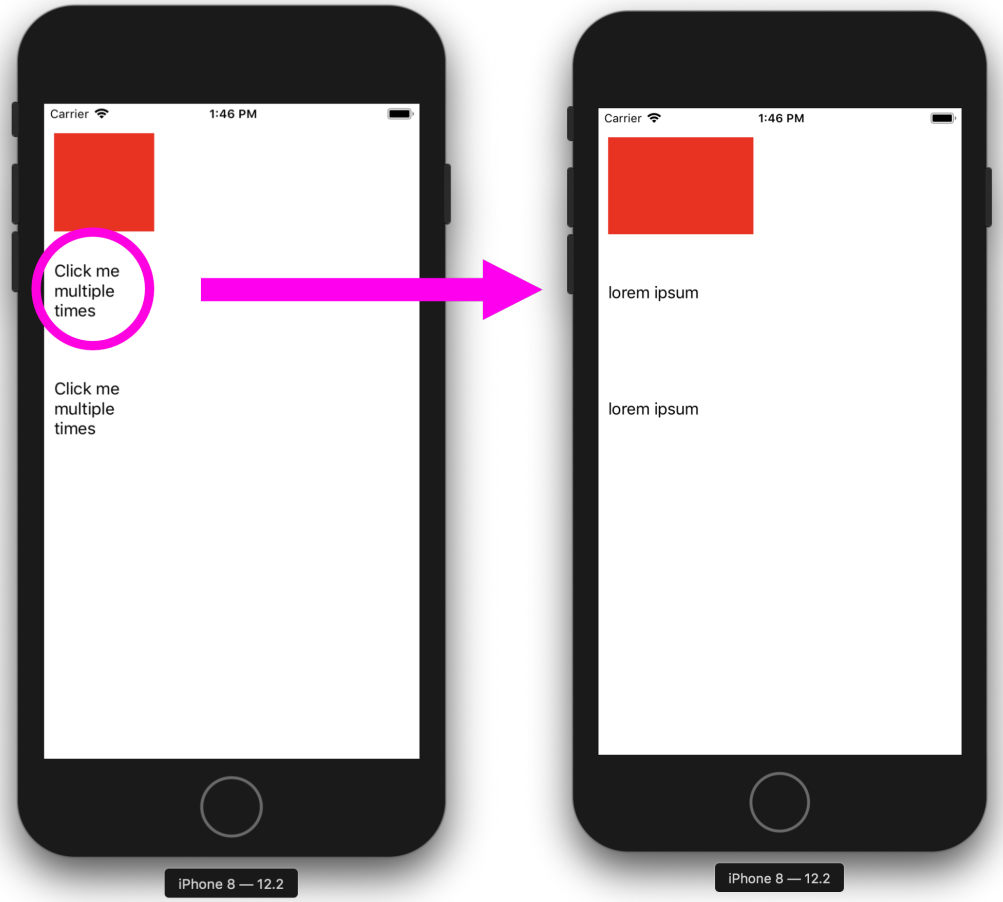

Figure 4.5: Demonstration of form data and frame change.

user tap is done, the changes need to get flushed. While the actions were executed, the value flag was inserted into the two TextLayer's dirt set. Also, the ColorLayer's dirt set includes the frame flag. Flushing the dirt now means, that there are only three changes that need to be applied, even though 7 changes happened:

1. TextLayer 1 changed text from "Click me multiple times" to "lorem ipsum", because of the Action.

2. TextLayer 2 changed text from "Click me multiple times" to "lorem ipsum", because of the shared form-data key.

3. TextLayer 1 changed text from "lorem ipsum" to "hallo welt", because of the Action.

4. TextLayer 2 changed text from "lorem ipsum" to "hallo welt", because of the shared form-data key.

5. TextLayer 1 changed text from "hallo welt" to "lorem ipsum", because of the Action.

6. TextLayer 2 changed text from "hallo welt" to "lorem ipsum", because of the shared form-data key.

7. ColorLayer changed it's frame, because of the Action.

The configuration shows, that the internal layer representations are calculating their values and frames, while the main thread stays interactive. To better visualize this behavior, every action will perform some artificial work to utilize the main thread for about one second.

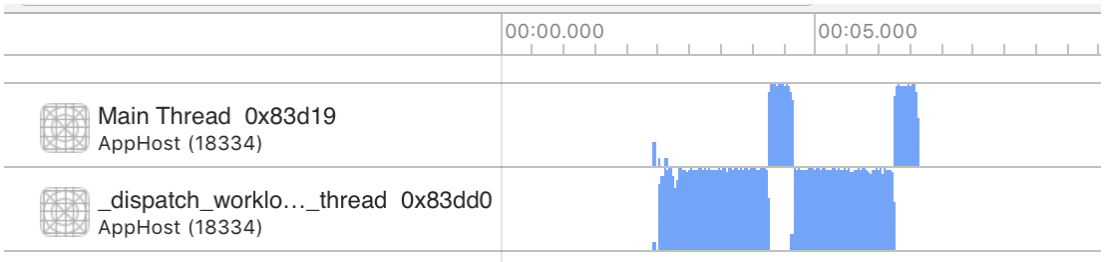

Figure 4.6: Thread utilization after user tapped the label two times

The graph of figure 4.6 shows thread utilization over time. The second thread is part of the iOS thread pool and can be assigned to multiple dispatch queue. The wide peak occurs, because there is artificial work executed on every action to simulate a fully configured app. While the actions are executed, the main thread is free and user interaction like scrolling the container view is still possible. After the virtual machine is done evaluating the resulting view hierarchy, the changes are send to the host, who applies them to the native views. Applying values to the native view is also extended with artificial workload to simulate an extensive app. Notice, that the VM thread waits for the UI to finish.

4.3.5 Testability

This approach allows black box testing, which is not possible with the current implementation. A black box test of the VM can launch an virtual app instance, tap layers and insert values into a text field or any other user interaction. All this happens by using the public VM interface, that would otherwise be used by the actual UI implementation. Those tests are platform independent and can run very fast on a Mac or inside a iOS host application. It is now possible to hard-code views, which then can be tested by simulating user input. This includes all of the special mos logic.

White box unit tests are also possible, since there are no public fields, but instead a app instance is passed to the sub components. Also, platform components like network or file system have to register to the VM by design. This makes injecting test implementations of those components straight forward. It is now possible to simulate an app and it's user input fully on any host system.

An example black box test of the demo app is outlined in listing 4.4. The test simulates one tap on the first text layer. Notice, that the VM implementation differs slightly from the previous code snippets, because *ReactiveSwift* was used. The core structure is still intact and the approach is implemented accordingly.

The test expects the color layer to increase it's width by 30 points, because of the *Grow* action. Also, it is expected that the two text layers will both have the value "lorem ipsum", because of the *ToggleText* action and the shared form data key.

The lines 2 to 26 setup the demo app. *XCTestExpectation* is a semaphore, that can be fulfilled. The test execution can be halted until an expectation is fulfilled by using *wait(for:timeout:)*. This way, the test host can wait for the VM to push it's first view. The simulation of the tap is executed in line 29. It is a call, that would normally be done by the native layer view of the UI implementation. Since layers are abstracted, anyone can tap a layer, even a test host. After the tap, the test hosts waits for the color layer to change it's frame. The test host uses the same interface, that would be used to update the native views frame. But instead, the host uses it to wait for the value to change, so the value can be checked. Finally, starting from line 34, the test host checks the color layer's frame and the values of the text layers.

Listing 4.4: Black box test of the test app configuration

```
1  func testApp() {
2    // Prepare waiting for first view to be pushed
3    let firstViewExpectation = XCTestExpectation()
4    let vmProxy = VirtualMachineDelegateProxy(firstViewExpectation:
         firstViewExpectation)
5
6    // Start the VM
7    let virtualMachine = VirtualMachine(delegate: vmProxy)
8    virtualMachine.boot(appConfig: AppConfig())
9
10   // Wait for first view to be pushed
11   wait(for: [firstViewExpectation], timeout: 30)
12
13   // Check layer count of first view
14   XCTAssertEqual(vmProxy.firstView?.layerInterfaces.count, 3)
15
16   // Check frame of the color layer
17   XCTAssertEqual(
```

```
18      (vmProxy.firstView?.layerInterfaces[0] as? ColorLayerInterface)?.
            frameProperty.value,
19      Rect(x: 10, y: 10, width: 100, height: 100)
20    )
21
22    // Prepare waiting for frame change of the color layer
23    let waitForFrameChange = XCTestExpectation()
24    vmProxy.firstView?.layerInterfaces[0].frameProperty.signal.
          observeValues { _ in
25      waitForFrameChange.fulfill()
26    }
27
28    // Trigger a tap on the text layer
29    vmProxy.firstView?.layerInterfaces[1].didTap()
30
31    // Wait for frame of the color layer to change
32    wait(for: [waitForFrameChange], timeout: 30)
33
34    // Check for color layer's frame to be 30 points wider
35    XCTAssertEqual(
36      vmProxy.firstView?.layerInterfaces[0].frameProperty.value,
37      Rect(x: 10, y: 10, width: 130, height: 100)
38    )
39
40    // Check for the values of the text layers
41    XCTAssertEqual(
42      (vmProxy.firstView?.layerInterfaces[1] as? TextLayerInterface)?.
            valueProperty.value,
43      "lorem ipsum"
44    )
45    XCTAssertEqual(
46      (vmProxy.firstView?.layerInterfaces[2] as? TextLayerInterface)?.
            valueProperty.value,
47      "lorem ipsum"
48    )
49  }
```

4.3.6 Core Rules

To prevent coding errors or inconsistencies, there are some rules that are arising out of it. Especially new features or changes must follow these rules, so the performance and maintainability gains are not lost.

1. **One user interaction must result in only one flush.**

Since this rule represents the key elements of the re-engineering approach, it is the most important one. There are a few exceptions in which there is UI functionality called right before a flush. En example is a pushNode action, which would push a new view of layers to the stack. After the layers are displayed, the remaining dirt is flushed. A pop of a view would result in first removing the view from the stack and then flushing changes. The changes of layers that don't exist anymore will not be flushed, because the flushing is done recursively and the layer's are not part of the tree anymore.

2. **No UI code is allowed inside the virtual machine module.**

   The virtual machine code should be able to compile on any Apple platform. The gold standard would be a platform independence, that allows the virtual machine code to build on Ubuntu. As a result, importing UIKit, CoreGraphics, CoreLocation, UserNotifications, EventKit, AVFoundation, MapKit and other modules are is not allowed. Only Foundation and other cross platform modules should be used.

3. **The callee of a public interface must make sure to continue on the correct dispatch queue**

   This helps the code to be more understandable, since random queue changes are harder to track. With this rule, a public interface can be used without the need to check the queue requirements. Queues should only be switched while crossing framework borders.

4. **Changes to internal layers that are relevant to the UI must result in the appropriate dirt flag to be set.**

   It is important for the internal logic implementation to strictly set the according flags on changes. The lay-outer must set the frame flag, but only if the layout frame actually changed. Sub-layered layers must set their sublayer flag, if they decide to reload their layer on a database change. Those are some examples of code positions, that need to be disciplined.

5. **VM APIs must be called using fire and forget.**

   To ensure a responsive UI, the VM must make sure to switch on the VM dispatch queue asynchronous. Since every callee must make sure to switch to the correct queue, the VM is responsible to perform asynchronous.

6. **UI APIs must be called synchronous, so the VM waits for the UI to finish.**

The UI make sure to respond synchronous, to make sure that it is safe to read the interface fields to update the UI. If the UI is asked to update some layers and some asynchronous task inside the VM is triggered that will write those properties, a thread collision is possible. A solution is to let the VM wait until the UI is done updating the layer.

7. **The public interface of the VM must be as thin as possible.**

   To prevent programming errors and inconsistencies, every public interface must be evaluated if it complies with the API contract, like the rule that results of interaction are applied by a flush. The VM must perform as much work as possible and delivers pre-rendered data to the UI. Pre-rendered data is data that has already been evaluated and is ready to be displayed.

8. **Platform interfaces are provided by the UI framework, but are run on the VM's dispatch queue.**

   For the VM to be fully testable, every interface to platform specific functionality like the file system, database, user defaults, screen size, location, network and push notifications are provided by delegates, that are registered in the VM. This way, the VM module can be fully black-box tested by unit tests. It is also possible to port the application to any platform. To prevent UI thread utilization, the interfaces are provided by the UI framework, but run on either the VM queue or a custom background queue. This way, tasks like querying the database does not block the main thread.

9. **No singletons or static objects.**

   To keep the code testable, no singletons or public static objects should be used. Instead, a app instance object, that represents one app instance should be passed to every sub-component. This makes it possible to run any amount of virtual apps in parallel, to enable fast testing. Also, it allows for dependency injection, to perform white box unit tests.

4.3.7 Maintainability

To make code maintainable and abstract the different logic levels, like configuration parsing, dynamic logic and interaction, some implementation rules must be set. The current implementation disregards all of the rules to some extend.

1. **Every component of the JSON model must be represented by a fully typed and read-only data structure.**

To prevent uncoordinated side-effects and a safe interface to the model, the text based model must be represented by a read-only data structure. This guarantees a strict separation of parsing logic, dynamic behavior and side effects. If a field of the model is considered mutable, a additional field is added in one of the dynamic logic classes, but not in the representation structure. This ensures that the time of applying the model to the mutable data is strictly defined and the original configuration is still in tact.

2. **Every interface between virtual machine and UI must be defined by a protocol[1].**

   Any implicit class API lacks comprehensibility, if it is spread across the class definition. To keep the interface clear, it should be defined as a custom protocol, so the intention and usage are explicit.

3. **Class dependencies must be defined by a protocol.**

   If a class depends on another class's functionality, a protocol should be defined, that an handle the classes requirements. This is important for test purposes, since a white box unit test must be able to simulate external behavior. The test host can pass a custom object, that implements the protocol, but simulates special behavior and values, to test the border cases of the class.

1 A protocol in Swift is similar to an interface in Java. It also allows the definition of public properties.

5 Evaluation

This chapter will take the implemented interpreter and the meta model of an app configuration and will classify the development approach. The properties of general models as described in section 2.3.1 will be compared to the meta model definition of mos. The comparison also includes the development approach of the meta model. The meta model and its instances can be examined for general validity and inconsistencies.

In the next section, the implementations of the interpreter are evaluated. This will be the focus, since the interpreter is the main subject in this thesis. The requirements as described in section 3.1 will be weight by individual comparison. The old and new implementations will then be rated based on their compliance with the requirements.

The evaluation is needed to be able to compare the two implementations and ultimately decide, if the new implementation is a step forward. Also, there is the question if it is needed to re-engineer the interpreter for a third time, based on the VM approach of section 4.3. Because the first approach was not perceived as a much better or faster client, a totally new approach has been considered, which is the VM structure.

The answers to the research question of section 1.4.2 will be backed by the result of the evaluation. This will then lead to the final conclusion.

5.1 Meta Model

This section will examine the meta model and will compare the approach of developing and evolving with common, established procedures. The meta model development is independent to its later runtime. This is why a meta model development for a interpreter can be compared to the development of meta models that will be used for code generation. This means, that the development approach of section 2.5.1 can be applied.

5.1.1 Development Approach

Since a model interpreter is not code centric, but focuses on features and requirements, the approach of 2.5.1 differs slightly to the one used for the development of the mos meta model.

The approach that was described in figure 2.4 begins by analyzing the application code of existing originals. Instead of analyzing source code, the requirements of generic mobile applications have been analyzed and divided into the three aspects. This was done, because no direct originals existed. Also, it is not necessary to develop an interpreter based on original code, but instead based on original abstract behavior.

Schematic repetitive code originally refers to code, that is abstracted into the model and later generated by a code generator. This can be applied to the code generator because it refers to the repetitive behavior of a mobile applications. It contains all the features that the application offers, that is described by the model. Some code could have been taken with minor changes from an original mobile application, because some design patterns of mobile applications also require a strong abstraction, so the UI code is generic anyway. However, some parts of the interpreter code is not found in application code, but in other reactive UI frameworks, because it allows the synchronization of values.

Generic code is domain specific code, that is used in all apps and is not part of the model. Since mobile application frameworks are already abstracting a lot of work, most of the code that is used in the interpreter is directly relevant for model interpretation features. One could also argue that all of the interpreter code is the generic code, since it might exist as a concrete version in an original.

Individual code can be interpreted as interpreter features, that have been developed for single specific applications. An example for that are tool layers, that display a ruler, a goniometer and a mechanic's level. Those layer have only been developed for a single app, that implements hardware store online services. This might not be custom code, that has been injected, but it is a model extension that was developed instead of providing a code interface. Custom code in the domain of platform independent applications is not goal-oriented, because the main idea is to reduce the need to write duplicated code in different languages.

5.1.2 Structure

The meta models structure was directly designed to be interpreted on mobile devices with fluctuating internet connectivity. That's why, all the components can be loaded

separately and be cached. Nodes, which define the basic view structure of an app, can be loaded lazy and on demand. The implicit behavior of the interpreter supports the models intentions, by operating cache first and loading only if the various hash values changed. This is done by calling the API's update interface which provides hashes for the different elements. For the model this means, that it's performance is directly depended on the web services that are delivering it. It is only a small part of a large infrastructure.

To evaluate the meta models platform dependence, all the features, that are directly referencing mobile devices will be outlined.

It is possible to define a nodes style based on the phone's device orientation and screen size. For screen size, the identifiers *phone*, *tab7* and *tab10* are available, which represent a normal phone, a seven inch tabled or a ten inch tablet. The device orientation can be distinguished by the values *all*, *portrait*, *landscape*, *portraitsensor*, *landscapesensor*.

Those configuration options are valid for any mobile device platform like tablets and phones. But it turned out that it can be also used directly for interactive digital signage devices, because they behave like a large tablet using their touch-based interface. Android and iOS devices are perfectly capable of mapping those identifiers to native values. A computer running desktop systems like Windows cannot directly provide those values, but instead can simulate a mobile device environment by always using *landscape* and *tab10* for example.

5.2 Architecture Analysis

The analysis of a software architecture is described in [Kle].

5.2.1 Native Approach

5.2.2 Virtual Machine Approach

5.3 Requirement Analysis

5.3.1 Native Approach

5.3.2 Virtual Machine Approach

		Async Display Kit			Native Approach		
		Globus	Mainz		Globus	Mainz	
		Catalog	Event List	Search	Catalog	Event List	Search
iPhone 5 (2012)	1	59,41 s	12,42 s	1,89 s	271,00 s	5,84 s	2,17 s
	2	55,26 s	12,15 s	1,58 s	267,00 s	5,98 s	2,07 s
	3	56,05 s	12,63 s	1,52 s	263,00 s	5,99 s	2,16 s
iPhone 6	1	19,27 s	4,47 s	0,89 s	63,00 s	4,09 s	1,05 s
	2	18,35 s	4,09 s	0,67 s	62,00 s	4,06 s	1,11 s
	3	17,96 s	4,38 s	0,74 s	60,00 s	4,22 s	0,99 s
iPhone X (2017)	1				19,51 s	2,97 s	0,76 s
	2				19,62 s	3,12 s	0,73 s
	3				19,48 s	3,42 s	0,81 s
Min		17,96 s	4,09 s	0,67 s	19,48 s	2,97 s	0,73 s
Max		59,41 s	12,63 s	1,89 s	271,00 s	5,99 s	2,17 s

Figure 5.1: Performance test

6 Conclusion

6.1 Summary

6.2 Discussion

6.3 Next Steps

6.3.1 Hybrid Approaches

6.3.2 Runtime Models

6.3.3 Abstraction

Bibliography

[Jea] Jeary, Sheridan; Fouad, Ali; Phalp, Keith und Kanyaru, John Mathenge: Embedding requirements within Model-Driven Architecture

[Kle] Klein, Mark; Kazman, Rick und Clements, Paul: *Evaluating Software Architectures*

[Mil] Miller, Joaquin und Mukerji, Jishnu: MDA Guide Version 1.0.1: S. 62

[Per] Perkins, Kleiner: Internet Trends Report 2018, URL https://www.kleinerperkins.com/perspectives/internet-trends-report-2018/

[Pop] Popek, Gerald J. und Goldberg, Robert P.: Formal Requirements for Virtualizable Third Generation Architectures. Bd. 17(7): S. 412–421, URL http://doi.acm.org/10.1145/361011.361073

[Sei] Seidewitz, Ed: What Models Mean. URL https://www.researchgate.net/publication/3248044_What_models_mean

[Staa] Stachowiak, Herbert: *Allgemeine Modelltheorie*, URL http://archive.org/details/Stachowiak1973AllgemeineModelltheorie

[Stab] Stahl, Thomas und Völter, Markus: *Model-driven software development: technology, engineering, management*, John Wiley, OCLC: ocm64453466

Index of abbreviations

AST Abstract Syntax Tree

API Application Programming Interface

CSS Cascading Style Sheets

DSL Domain Specific Language

DOM Document Object Model

HTML Hyper Text Markup Language

MDA Model Driven Architecture

MDD Model Driven Development

MDSD Model Driven Software Development

mos Mobility Suite

OOP Object Oriented Programming

HTTP Hypertext Transfer Protocol

UI User Interface

GUI Graphical User Interface

PHP PHP: Hypertext Preprocessor, earlier: Personal Home Page

UML Unified Modeling Language

URL Unified Resource Locator

XSLT Extensible Stylesheet Language Transformations

JSON JavaScript Object Notation

SQL Structured Query Language

RISC Reduced Instruction Set Computer

VM Virtual Machine

JVM Java Virtual Machine

SPM Swift Package Manager

XML Extensible Markup Language

XHTML Extensible Hypertext Markup Language

OMG Object Management Group

PIM Platform Independent Model

PSM Platform Specific Model

VCS Version Control System

List of Figures

1.1 Display notch with safe area. 2

2.1 Gießen App, Marketing showcase image. Source: Giessen Entdecken http://giessen-entdecken.de/die-app/ 6
2.2 City marketing stele for shapping streets. Source: Fabrik19 Portfolio https://fabrik19.de/project/interaktive-stele-giessener-seltersweg/?lang=en 7
2.3 The web based node editor is used to develop and edit the models. The Lauterbach App landing page is displayed. 8
2.4 Analyzing an existing implementation (according to [Stab] 15). 15
2.5 Separation of code into source code and steady code (according to [Stab] 15). 16
2.6 API based code generator infrastructure 21
2.7 Interpreter infrastructure for common code interpreters 22

3.1 Iterative development 31
3.2 Compilation of development artifacts for mos 32
3.3 Virtual machine, to separate mos from platform code 37
3.4 Change management cycle 39

4.1 Basic interaction between VM and UI 45
4.2 Simulation of infinite scrolling by resetting the window at the boundaries. 45
4.3 Synchronization of VM and UI 50
4.4 Project structure of a proof of concept implementation 50
4.5 Demonstration of form data and frame change. 53
4.6 Thread utilization after user tapped the label two times 54

5.1 Performance test 64

List of Tables

3.1 Events, that are used for change detection by Angular and their mos counterpart. 38

4.1 Interactions, that produce dirt, that needs to be flushed. 47

Listings

2.1 data.xml - The model that is data. 18
2.2 template.xslt - The templates that transforms data into HTML. 18
2.3 XSLT ouput. 19
2.4 Using an API to generate Java code, written in Swift 20
2.5 Java code result of lising 2.4 20

4.1 Layer class (excerpt) 42
4.2 Key data structures of dirt and flush 47
4.3 Demo configuration 52
4.4 Black box test of the test app configuration 55

.